

NaN (Need a Name)

1 General

1.1 Plantilla

```
1 #define FIN ios::sync_with_stdio(0);cin.tie(0);
   cout.tie(0)
```

1.2 Compilar

```
1 g++ -std=c++17 -DLOCAL -g -O2 -Wconversion -
   Wshadow -Wall -Wextra -D_GLIBCXX_DEBUG -c %f
2 g++ -std=c++17 -DLOCAL -g -O2 -Wconversion -
   Wshadow -Wall -Wextra -D_GLIBCXX_DEBUG -o %e
   %f
3
4 code ~/.bashrc # archivo para crear alias
5
6 compile() {
7     g++ "$1.cpp" -o "$1" 2>&1|grep "error:"
8 } # Crea ejecutable con mismo nombre que el
   archivo .cpp
9
10 source ~/.bashrc # resetear el terminal para
   aplicar cambios
11 ulimit -s 1048576 # aumentar el stack a 1Gb (
   Ubuntu)
```

2 Data Structures

2.1 Compresion De Coordenadas

```
1 struct compresion {
2     vector<int> todos;
3     compresion(vector<int> v) {
4         todos = v; sort(all(todos));
5         todos.erase(unique(all(todos)),todos.end
           ());
6     }
7     int obtener(int x) { return (int)(lower_bound
           (all(todos),x)-todos.begin()); }
8 };
```

2.2 Dsu

```
1 struct DSU {
2     vi link, sz;
3     DSU(int tam) {
4         link.resize(tam+5), sz.resize(tam+5);
5         forn(i, tam+5) link[i] = i, sz[i] = 1;
6     }
7     ll find(ll x){ return link[x] = (link[x] == x
           ? x : find(link[x])); }
8     bool same(ll a, ll b) { return find(a) ==
           find(b); }
9     void join(ll a, ll b) {
10         a = find(a), b = find(b);
11         if(a == b) return;
12         if(sz[a] < sz[b]) swap(a,b);
13         sz[a] += sz[b];
14         link[b] = a;
15     }
16 };
```

2.3 Implicit Treap

```
1 typedef struct item *pitem;
2 struct item {
3     ll key, prior, cont, mini, sum;
4     bool rev; // (parameters for lazy prop)
5     pitem l, r;
6     item(ll key):
7         key(key),prior(rand()),cont(1),l(NULL),r(
           NULL),rev(0) {}
8 };
9 void push(pitem it) { //Lazy for reverse array
10     if(it && it->rev) {
11         swap(it->l,it->r);
12         if(it->l)it->l->rev ^= true;
13         if(it->r)it->r->rev ^= true;
14         it->rev = false;
15     }
16 }
17 int cont(pitem it) {return it ? it->cont : 0;}
18 ll mini(pitem it) {return it ? it->mini : 1<<30;}
19 ll sum(pitem it) {return it ? it->sum : 0LL;}
20 void upd_cont(pitem it){
21     if(it) {
22         it->cont = cont(it->l) + cont(it->r) + 1;
23         it->mini = min(it->key,min(mini(it->l),
           mini(it->r)));
24         it->sum = it->key + sum(it->l) + sum(it->
           r);
25     }
26 }
27 void merge(pitem &t, pitem l, pitem r) { //Merge
   l and r, new root t
28     push(l); push(r);
29     if(!l || !r) t = l ? l : r;
30     else if(l->prior > r->prior) merge(l->r,l->r,
           r), t=l;
31     else merge(r->l,l,r->l), t = r;
32     upd_cont(t);
33 }
34 void split(pitem t, pitem& l, pitem& r, int sz){
   //sz:desired size of l
35     if(!t) { l = r = 0; return;}
36     push(t);
37     if(sz <= cont(t->l)) split(t->l,l,t->l,sz), r
           = t;
38     else split(t->r,t->r,r,sz-1-cont(t->l)), l =
           t;
39     upd_cont(t);
40 }
41 int find_min(pitem t) { //Devuelve la posicion
   del minimo
42     push(t);
43     if(t->mini == t->key) return cont(t->l);
44     else if(t->l && t->l->mini == t->mini) return
           find_min(t->l);
45     else return cont(t->l)+1+find_min(t->r);
46 }
47 void set_treap(pitem &root, int p, ll val) {
48     pitem aux1 = NULL, aux2 = NULL;
49     split(root,root,aux2,p+1); split(root,root,
           aux1,p);
50     aux1->key = val;
51     merge(root,root,aux1); merge(root,root,aux2);
52 }
```

```

53 ll get_sum(pitem &root, int l, int r) {
54     pitem aux1 = NULL, aux2 = NULL;
55     split(root,root,aux2,r+1); split(root,root,
56         aux1,l);
57     ll ans = aux1->mini; // aux1->mini for
58         minimum
59     merge(root,root,aux1); merge(root,root,aux2);
60     return ans;
61 }
62 void DFS(pitem t) { //Good for debug
63     DBG(t->key);
64     if(t->l != NULL) DBG(t->l->key); if(t->r !=
65         NULL) DBG(t->r->key);
66     RAYA;
67     if(t->l != NULL) DFS(t->l); if(t->r != NULL)
68         DFS(t->r);
69 }

```

2.4 Indexed Set

```

1 using namespace __gnu_pbds;
2 typedef tree<ll,null_type,less<ll>,rb_tree_tag,
3     tree_order_statistics_node_update>
4     indexed_set;
5 indexed_set s; s.insert(2); s.insert(3); s.insert
6     (5);
7 auto x=s.find_by_order(2); // retorna el valor en
8     la posicion 2 (5)
9 int pos=int(s.order_of_key(3)); // retorna la
10     posicion que se encuentra el 3 (1)

```

2.5 Lazy-Propagation

```

1 typedef long long tipo;
2 const int NEUT = 0; // REMINDER !!!
3 struct node {
4     tipo ans, l, r, lazy = 0;
5     bool upd = false;
6     node() { ans = lazy = 0; upd = false; l = r =
7         -1; } // REMINDER !!! SET NEUT
8     node(tipo val, int pos) : ans(val), l(pos), r(
9         pos) {} // Set node
10     void set_lazy(tipo x) { lazy += x; upd = true;
11     }
12 };
13 struct segtree_lazy {
14     #define l(x) int(x<<1)
15     #define r(x) int(x<<1|1)
16     vector<node> t; int tam;
17     node op(node a, node b) {
18         node aux; aux.ans = a.ans + b.ans; //
19             Operacion de query
20         aux.l = a.l; aux.r = b.r;
21         return aux;
22     }
23     void node_update(node &cur) {
24         cur.ans += cur.lazy * (cur.r-cur.l+1); //
25             Operacion update
26     }
27     void reset_lazy(node &cur) {
28         cur.lazy = 0; cur.upd = false; //Poner el
29             neutro del update
30     }
31     void push(int p) {
32         node &cur = t[p];
33         if(cur.upd == true) {

```

```

28         node_update(cur);
29         if(cur.l < cur.r) {
30             t[l(p)].set_lazy(cur.lazy);
31             t[r(p)].set_lazy(cur.lazy);
32         }
33         reset_lazy(cur);
34     }
35 }
36 node query(int l, int r, int p = 1) {
37     push(p); node &cur = t[p];
38     if(l > cur.r || r < cur.l) return node(); //
39         Return NEUT
40     if(l <= cur.l && cur.r <= r) return cur;
41     return op(query(l,r,l(p)),query(l,r,r(p)));
42 }
43 void update(int l, int r, tipo val, int p = 1)
44     { // root at p = 1
45     push(p); node &cur = t[p];
46     if(l > cur.r || r < cur.l) return;
47     if(l <= cur.l && cur.r <= r) {
48         cur.set_lazy(val); push(p); return;
49     }
50     update(l, r, val, l(p)); update(l, r, val, r(
51         p));
52     cur = op(t[l(p)], t[r(p)]);
53 }
54 void build(vector<tipo> v, int n) { //
55     iterative build
56     tam = sizeof(int) * 8 - __builtin_clz(n); tam
57         = 1<<tam;
58     t.resize(2*tam); v.resize(tam);
59     forn(i,tam) t[tam+i] = node(v[i],i);
60     for(int i = tam - 1; i > 0; i--) t[i] = op(t[
61         l(i)],t[r(i)]);
62 }
63 };

```

2.6 Mo Dquery

```

1 struct query{
2     int l, r, ind;
3     bool operator <(query q) const {
4         return r < q.r;
5     }
6 };
7 struct mo {
8     const int block = 448; // sqrt(MAXN)
9     vector<vector<query>> q;
10     vector<ll> ans, cont;
11     mo(){
12         ans.resize(MAXN), cont.resize(MAXN);
13         q.resize(MAXN/block+5);
14     }
15     void read_queries(int m) { // m --> number of
16         queries
17         ans.resize(m,0);
18         forn(i, m) {
19             int l, r; cin >> l >> r;
20             l--; r--; // For 0-indexed
21             q[l / block].pb({l,r,(int)i});
22         }
23     }
24     void add(ll &sum, ll val) {
25         if(cont[val] == 0) sum++;
26         cont[val]++;

```

```

26     }
27     void erase(ll &sum, ll val) {
28         if(cont[val] == 1) sum--;
29         cont[val]--;
30     }
31     void get_mo(vector<ll> &v) {
32         ll sum = 0;
33         forn(i,q.size()) {
34             sort(all(q[i]));
35             int l, r; l = r = block * i;
36             for(query u : q[i]) { // Solve for [l
37                                     ,r]
38                 while(r <= u.r) add(sum,v[r]), r
39                     ++;
40                 while(l <= u.l) erase(sum,v[l]),
41                     l++;
42                 while(l > u.l) l--, add(sum,v[l])
43                     ;
44                 ans[u.ind] = sum;
45             }
46         }
47     }
48 };

```

2.7 Persistent

```

1 struct Vertex {
2     Vertex *l, *r;
3     ll sum;
4     Vertex(ll val) : l(nullptr), r(nullptr), sum(
5         val) {}
6     Vertex(Vertex *l, Vertex *r) : l(l), r(r),
7         sum(0) {
8         if (l) sum += l->sum;
9         if (r) sum += r->sum;
10    }
11 };
12 Vertex* build(ll a[], ll tl, ll tr) {
13     if (tl == tr)
14         return new Vertex(a[tl]);
15     ll tm = (tl + tr) / 2;
16     return new Vertex(build(a, tl, tm), build(a,
17         tm+1, tr));
18 }
19 ll get_sum(Vertex* v, ll tl, ll tr, ll l, ll r) {
20     if (l > r)
21         return 0;
22     if (l == tl && tr == r)
23         return v->sum;
24     ll tm = (tl + tr) / 2;
25     return get_sum(v->l, tl, tm, l, min(r, tm))
26         + get_sum(v->r, tm+1, tr, max(l, tm+1),
27             r);
28 }
29 Vertex* update(Vertex* v, ll tl, ll tr, ll pos,
30     ll new_val) {
31     if (tl == tr)
32         return new Vertex(new_val);
33     ll tm = (tl + tr) / 2;
34     if (pos <= tm)
35         return new Vertex(update(v->l, tl, tm,
36             pos, new_val), v->r);
37     else
38         return new Vertex(v->l, update(v->r, tm

```

```

+1, tr, pos, new_val));
33 }
34 vector <Vertex*> estado;

```

2.8 Segtree-Iterative

```

1 typedef long long tipo;
2 struct segtree {
3     vector <tipo> t; int tam;
4     tipo NEUT = 0;
5     tipo op(tipo a, tipo b){ return a+b; }
6     void build(vector<tipo>v, int n) {
7         tam = sizeof(int) * 8 - __builtin_clz(n); tam
8             = 1<< tam;
9         t.resize(2*tam,NEUT); forn(i, n) t[tam+i] = v
10             [i];
11         for(int i = tam - 1; i > 0; i--) t[i] = op(t[
12             i<<1], t[i<<1|1]);
13     }
14     void update(int p, tipo value) {
15         for (t[p += tam] = value; p > 1; p >>= 1) t[p
16             >>1] = op(t[p], t[p^1]);
17     }
18     tipo query(int l, int r) {
19         tipo res = NEUT;
20         for (l += tam, r += tam; l <= r; l >>= 1, r
21             >>= 1) {
22             if (l&1) res = op(res, t[l++]);
23             if (!(r&1)) res = op(res, t[r--]);
24         }
25         return res;
26     }
27 };

```

2.9 Segtree 2D

```

1 struct st2d{ // 0 indexado, [l, r]
2     // PARA HACER BUILD HAY QUE HACER LOS N*N
3     updates
4     int n; vector<vi> st; ll NEUT=0;
5     ll op(ll a, ll b){return a+b;}
6     st2d(int _n): n(_n) { st.resize(2*n+5, vi(2*n
7         +5, 0)); }
8     void upd(int x, int y, ll v){
9         st[x+n][y+n]=v;
10        for(int j=y+n;j>1;j>>=1)st[x+n][j>>1]=op(st[x
11            +n][j],st[x+n][j^1]);
12        for(int i=x+n;i>1;i>>=1)for(int j=y+n;j;j
13            >>=1) st[i>>1][j]=op(st[i][j],st[i^1][j])
14            ;
15    }
16    ll query(int x0, int x1, int y0, int y1){
17        ll r=NEUT; x1++, y1++;
18        for(int i0=x0+n,i1=x1+n;i0<i1;i0>>=1,i1>>=1){
19            int t[4],q=0;
20            if(i0&1)t[q++]=i0++;
21            if(i1&1)t[q++]--i1;
22            forn(k,q) for(int j0=y0+n,j1=y1+n;j0<j1;j0
23                >>=1,j1>>=1){
24                if(j0&1)r=op(r,st[t[k]][j0++]);
25                if(j1&1)r=op(r,st[t[k]][--j1]);
26            }
27        }
28        return r;
29    }
30 };

```

3 Dp

3.1 Cht Dynamic

```

1 typedef ll tc;
2 const tc is_query = -(1LL << 62);
3 struct Line {
4     tc m, b;
5     mutable multiset<Line>::iterator it, end;
6     const Line* succ(multiset<Line>::iterator it)
7         const {
8         return (++it == end ? NULL : &*it);
9     }
10    bool operator<(const Line& rhs) const {
11        if (rhs.b != is_query) return m < rhs.m;
12        const Line *s = succ(it);
13        if (!s) return 0;
14        return b - s->b < (s->m - m) * rhs.m;
15    }
16}; // Para minimo, cambiar el signo de m y b.
17struct HullDynamic : public multiset<Line> {
18    bool bad(iterator y) { // for maximum
19        iterator z = next(y);
20        if (y == begin()) {
21            if (z == end()) return false;
22            return y->m == z->m && y->b <= z->b;
23        }
24        iterator x = prev(y);
25        if (z == end()) return y->m == x->m && y->b <= x->b;
26        ;
27        return (x->b - y->b) * (z->m - y->m) >= (y->b - z->b) *
28            (y->m - x->m);
29    }
30    iterator next(iterator y) { return ++y; }
31    iterator prev(iterator y) { return --y; }
32    void add(tc m, tc b) {
33        // m *= -1; b *= -1;
34        iterator y = insert((Line){m, b});
35        y->it = y; y->end = end();
36        if (bad(y)) { erase(y); return; }
37        while (next(y) != end() && bad(next(y))) erase(
38            next(y));
39        while (y != begin() && bad(prev(y))) erase(prev(
40            y));
41    }
42    tc eval(tc x) {
43        Line l = *lower_bound((Line){x, is_query});
44        return l.m * x + l.b; // -1 * (l.m * x + l.b)
45    }
46};

```

3.2 Divide And Conquer Dp Opt

```

1 ll costo[MAXN][MAXN];
2 vector<ll> last(MAXN), dp(MAXN);
3 void calc_costo(int n, vector<ll> &v) {
4     void compute(int l, int r, int optl, int optr) {
5         if (l > r) return;
6         int med = (l + r) / 2;
7         pair<ll, ll> best = {INF, -1};
8         for (int p = optl; p <= min(med, optr); p++) {
9             best = min(best, {last[p] + costo[p + 1][med
10                 ], p});
11         }
12         dp[med] = best.first;
13         int opt = best.second;

```

```

13         compute(l, med - 1, optl, opt);
14         compute(med + 1, r, opt, optr);
15     }
16     ll solve_dac(int n, int k) {
17         for (int i = 0; i < n; i++) last[i] = costo
18             [0][i];
19         for (int i = 2; i <= k; i++) {
20             fill(all(dp), INF);
21             compute(0, n - 1, 0, n - 1);
22             last = dp;
23         }
24         return dp[n - 1];
25     }

```

3.3 Knapsack Weighted

```

1 vector<bool> knapsack_weighted(vector<int> v,
2     int n) {
3     int sum = accumulate(all(v), 0LL);
4     vector<bool> dp(sum + 1, false);
5     dp[0] = true;
6     sort(all(v));
7     vector<pair<int, int>> num; // (num, #count)
8     int count = 1, last = v[0];
9     for (i = 1; i < n; i++) {
10         if (v[i] == last) count++;
11         else {
12             num.pb({last, count});
13             last = v[i];
14             count = 1;
15         }
16     }
17     num.pb({last, count});
18     for (auto [x, c] : num) {
19         vector<bool> next_dp = dp;
20         for (i = x; i < sum; i++) {
21             int can = 0;
22             for (int j = i; j <= sum; j += x) {
23                 next_dp[j] = next_dp[j] | (can > 0);
24                 if (dp[j]) can++;
25                 if (j - c * x >= 0 and dp[j - c * x]) can--;
26             }
27         }
28         dp = next_dp;
29     }
30     return dp;
31 }

```

3.4 Knuth Division Dp Opt

```

1 const ll INF = (1LL) * (1e18 + 10);
2 const int MAXN = 5005;
3 vector<ll> prefix;
4 // Sea Pos(i, j) la posicion del corte que
5 // optimiza la dp, intervalo [i, j]
6 // Se puede aplicar optimizacion de Knuth si se
7 // cumple que:
8 // Pos(i, j - 1) <= pos(i, j) <= pos(i + 1, j)
9 // Objetivo: llevar un array a elementos
10 // individuales mediante cortes.
11 ll suma(int i, int j) {
12     return prefix[j + 1] - prefix[i];
13 } // Suma de los elementos del inter
14 ll solve_Knuth(int n) {
15     vector<ll> pos(n + 1);

```

```

13     vector <vector <ll> > dp(MAXN,vector<ll>(MAXN
14         ,INF));
15     for(int i = 0; i < n; i++) dp[i][i] = 0, pos[
16         i] = i;
17     for(int len = 1; len < n; len++) {
18         vector <ll> aux(n+1,0);
19         for(int i = 0; i < n - len; i++) {
20             for(int k = pos[i]; k <= pos[i+1]; k
21                 ++){
22                 ll cost = suma(i,i+len) + dp[i][k
23                     ] + dp[k+1][i+len];
24                 if(cost < dp[i][i+len]) {
25                     dp[i][i+len] = cost; aux[i] =
26                         k;
27                 }
28             }
29         }
30         pos = aux;
31     }
32     return dp[0][n-1];
33 }

```

3.5 Lis

```

1 ll lis(vi &a, int strict = 0){
2     vi temp; temp.pb(a[0]);
3     forr(i, 1, SZ(a)){
4         ll x = a[i];
5         if(x >= temp.back()+strict) temp.pb(x);
6         else { auto it = upper_bound(all(temp), x-
7             strict); *it = x; } }
8     return SZ(temp); }

```

4 Geometry

4.1 Angular Sort

```

1 // Angular sort
2 punto origin = {0LL,0LL};
3 bool AngularSort(const punto &A, const punto &B)
4     {
5         bool aorig = (A < origin); // semiplano
6         izquierdo o derecho
7         bool borig = (B < origin); // semiplano
8         izquierdo o derecho
9         if (aorig != borig) { return aorig > borig; }
10        // primero sem izq
11        return (A^B) > 0; // desempate por producto
12        vectorial si mismo sem
13    }

```

4.2 Calipers

```

1 // Devuelve la distancia entre los puntos mas
2 // lejanos en O(N)
3 // Cuando se usa con nuestra convex_hull, hacer
4 // reverse porque vienen CW
5 ld callipers(vector<punto> p, int n){
6     ld r=0; // prereq: Convex, ccw, NO
7     COLLINEAR POINTS
8     for(int i=0,j=n-2?0:1;i<j;++i){
9         for(;;j=(j+1)%n){
10             r=max(r,(p[i]-p[j]).mod());
11             if((p[(i+1)%n]-p[i])^(p[(j+1)%n]-p[j
12                 ]))<=EPS)break;
13         }
14     }
15 }

```

```

10     }
11     return r;
12 }

```

4.3 Closest Points Monogon

```

1 const ll INF = (ll)(1e15+10); // (1e18+10)
2 template<typename T>
3 using minpq = priority_queue<T, vector<T>,
4     greater<T>>;
5 struct punto {
6     ll x, y;
7     punto (ll x = 0, ll y = 0) : x(x), y(y) {}
8     punto operator -(const punto &p) const {
9         return punto(x - p.x, y - p.y); }
10    ll len2() const { return x * x + y * y; }
11    bool const operator <(const punto &p) const {
12        return (x != p.x) ? x < p.x : y < p.y;
13    }
14 };
15 istream& operator>>(istream &is, punto &p) {
16     return is >> p.x >> p.y;
17 }
18 ll closest(vector<punto> &p, int l, int r) { //
19     Init l = 0 and r = p.size()-1
20     if(l == r) return INF;
21     int m = (l + r) / 2;
22     ll mid = p[m].x;
23     ll d = min(closest(p, l, m), closest(p, m +
24         1, r));
25     vector<punto> A, B, C;
26     forr(i, l, m + 1) {
27         ll r = mid - p[i].x;
28         if(r * r <= d) A.push_back(p[i]);
29     }
30     forr(i, m + 1, r + 1) {
31         ll r = mid - p[i].x;
32         if(r * r <= d) B.push_back(p[i]);
33     }
34     auto cmpy = [&](punto a, punto b) { return a.
35         y < b.y; };
36     merge(all(A), all(B), back_inserter(C), cmpy)
37     ;
38     forr(i, 0, C.size()) {
39         int j = i + 1;
40         while(j < C.size() && (C[i] - C[j]).y * (
41             C[i] - C[j]).y <= d) {
42             d = min(d, (C[i] - C[j]).len2());
43             j++;
44         }
45     }
46     inplace_merge(p.begin() + l, p.begin() + m +
47         1, p.begin() + r + 1, cmpy);
48     return d;
49 }
50 void sorting(vector<punto> &p) { sort(all(p)); }
51 // Don't Forget

```

4.4 Convex Hull

```

1 const ld EPS = 1e-10;
2 void convex_hull(vector<punto> &a) {
3     if(SZ(a) == 1) return;
4     sort(all(a));
5     punto p1 = a[0];
6     vector<punto> up, down;

```

```

7   up.pb(p1); down.pb(p1);
8   forr(i,1,SZ(a)) {
9       int n = SZ(up), m = SZ(down);
10      while(n > 1 && ((up[n-1]-up[n-2])^(a[i]-
11          up[n-1])) >= -EPS) {
12          up.pop_back(); n--;
13      } up.pb(a[i]);
14      while(m > 1 && ((down[m-1]-down[m-2])^(a[
15          i]-down[m-1])) <= EPS) {
16          down.pop_back(); m--;
17      } down.pb(a[i]);
18  } // Cambiar EPS a 0 para mejor precision en
19      enteros.
20  a.clear();
21  for(punto u : up) a.pb(u);
22  for(int i = SZ(down)-2; i > 0; i--) a.pb(down
23      [i]);
24  }

```

4.5 Formulas

```

1  // Shoelace formula
2  tipo area(vector <punto> &v) {
3      tipo ans = 0.0; int n = v.size();
4      forn(i,n) ans += v[i] ^ v[(i+1)%n];
5      return fabs(ans/2.0);
6  }
7  tipo find_alpha(recta r, punto p) {
8      return r.v.x != 0 ? (p.x-r.p.x)/r.v.x : (p.y-
9          r.p.y)/r.v.y;
10 }

```

4.6 Intersection All

```

1  struct recta { // Puede usarse para segmentos ([p
2      ,p+v] o alpha = [0,1])
3      punto v, p; // v -> director, p -> punto por
4          donde pasa
5      recta(punto p1, punto p2) { v = (p2-p1); p =
6          p1;}
7      recta() {}
8      recta(tipo A, tipo B, tipo C) { // Transform
9          Ax + By + C = 0
10         v = {-B,A}; A != 0 ? p = {-C / A,0} : p =
11             {0,-C / B};
12     }
13     bool is_in(punto q){return fabs((q.x-p.x)*v.y
14         - (q.y-p.y)*v.x) < EPS;}
15     punto eval(double x) {return x * v + p;}
16 };
17 bool inter_recta(recta &r1, recta &r2, punto &ans
18 ) {
19     // Retorna false si son paralelas, sino
20     guarda el punto en ans
21     if(abs(r1.v ^ r2.v) < EPS) return false;
22     tipo alpha = tipo((r2.p - r1.p)^r2.v) / tipo
23         (r1.v^r2.v);
24     ans = r1.p + alpha * r1.v;
25     return true;
26 }
27 bool inter_seg(recta &r1, recta &r2, punto &ans)
28 {
29     if(r1.p == r2.p || r1.p == r2.p+r2.v) {ans =
30         r1.p; return true;}
31     if(r1.p+r1.v == r2.p || r1.p+r1.v == r2.p+r2.
32         v) {

```

```

33         ans = r1.p+r1.v; return true; } //Casos
34         que coincidan extremos
35     if(abs(r1.v ^ r2.v) < EPS) return false; //
36         son paralelos
37     tipo alpha = tipo((r2.p - r1.p)^r2.v) / tipo
38         (r1.v^r2.v);
39     tipo beta = tipo((r1.p - r2.p)^r1.v) / tipo(
40         r2.v^r1.v);
41     if(alpha < -EPS || beta < -EPS) return false;
42     if(alpha > 1.0+EPS || beta > 1.0+EPS) return
43         false;
44     ans = r1.p + alpha * r1.v; return true;
45 }
46 struct circ { punto c; tipo r; };
47 tipo dist_point_line(punto &p, recta &r) {
48     punto p1 = r.p, p2 = r.p + r.v;
49     return fabs((p1-p)^r.v)/r.v.mod();
50 }
51 punto project(punto a, punto b) { //Proyeccion de
52     b sobre a
53     return ((a*b)/a.mod2()) * a;
54 }
55 vector<punto> inter_circ_line(recta r, circ c) {
56     vector <punto> ans; tipo dist =
57         dist_point_line(c.c,r);
58     if(dist > c.r+EPS) return ans;
59     (c.c-r.p) * r.v != 0 ? r.p = r.p : r.p = r.p
60         + r.v;
61     punto aux = c.c - r.p, dir = project(r.v,aux)
62         ;
63     if(fabs(dist-c.r) <= EPS) {ans.pb(r.p + dir);
64         return ans;}
65     tipo factor = sqrt(c.r*c.r - dist*dist)/dir.
66         mod();
67     ans.pb(r.p + dir + factor * dir); ans.pb(r.p
68         + dir - factor * dir);
69     return ans;
70 }
71 tipo intersection_area(circ a, circ b) {
72     punto aux = (b.c - a.c); tipo dist=aux.mod()
73         , dist2=aux.mod2();
74     if(a.r + b.r - dist < -EPS) return 0;
75     if(fabs(a.r - b.r) - dist > -EPS) {
76         return min(a.r, b.r) * min(a.r, b.r) * 2*
77             acosl(0); }
78     tipo alpha = acosl((dist2 + a.r*a.r - b.r*b.
79         r) / (2 * dist * a.r));
80     tipo beta = acosl((dist2 + b.r*b.r - a.r*a.r
81         ) / (2 * dist * b.r));
82     tipo ans1 = (alpha - sinl(alpha+alpha)*0.5)
83         * a.r * a.r;
84     tipo ans2 = (beta - sinl(beta+beta)*0.5) * b
85         .r * b.r;
86     return ans1 + ans2;
87 }
88 vector<punto> inter_circ_circ(circ a, circ b) {
89     vector <punto> ans;
90     if (a.c == b.c) {
91         return abs(a.r-b.r) <= EPS ? vector<punto
92             >{a.c,a.c,a.c} : ans;
93     }
94     b.c = b.c - a.c; punto aux = a.c; a.c = a.c -
95         a.c;
96     recta r(-2*b.c.x, -2*b.c.y, a.r*a.r - b.r*b.r
97         + b.c.x*b.c.x + b.c.y*b.c.y);

```

```

65     ans = inter_circ_line(r,a);
66     forn(i,ans.size()) ans[i] = ans[i] + aux;
        return ans;
67 }
68 void tangents(punto c, double r1, double r2,
        vector<recta> & ans) {
69     double r = r2 - r1;
70     double z = c.xc.x + c.yc.y;
71     double d = z - rr;
72     if (d < -EPS) return;
73     d = sqrt(abs(d));
74     recta l((c.x r + c.y * d) / z, (c.y * r - c.x
        * d) / z, r1);
75     ans.push_back(l);
76 }
77 vector<recta> tangents(circ a, circ b) {
78     vector<recta> ans;
79     for (int i=-1; i<=1; i+=2) for (int j=-1; j
        <=1; j+=2) tangents(b.c-a.c, a.ri, b.rj,
        ans);
80     forn(i,SZ(ans)) ans[i].p =ans[i].p + a.c;
81     return ans;
82 }

```

4.7 Point In Poly

```

1 bool point_in_poly(vector <punto> &v, punto p) {
    // O(n) for non-convex
2     unsigned i, j, mi, mj, c = 0;
3     for(i = 0, j = v.size()-1; i < v.size(); j =
        i++) {
4         if((v[i].y <= p.y && p.y < v[j].y) || (v[
            j].y <= p.y && p.y < v[i].y)) {
5             mi = i; mj = j; if(v[mi].y > v[mj].y)
                swap(mi,mj);
6             if(((p-v[mi]) ^ (v[mj]-v[mi])) < 0 )
                c ^= 1;
7         }
8     }
9     return c;
10 }
11 bool point_in_poly_convex(vector <punto> &v,
    punto p) { // O(log n)
12     bool ans = true;
13     if( ((v[1]-v[0]) ^ (v[2]-v[1])) >= 0) reverse
        (all(v));
14     int a = 1, b = v.size()-1;
15     while(b-a > 1) {
16         int med = (a+b)/2;
17         if( ((v[med]-v[0]) ^ (p-v[med])) <= 0 ) a
            = med;
18         else b = med;
19     }
20     if( ((v[a]-v[0]) ^ (p - v[a])) > 0) ans =
        false;
21     if( ((v[b]-v[a]) ^ (p - v[b])) > 0) ans =
        false;
22     if( ((v[0]-v[b]) ^ (p - v[0])) > 0) ans =
        false;
23     return ans;
24 }

```

4.8 Template Punto

```

1 typedef long double tipo; //Cambiar a long long
    para operar en enteros

```

```

2 double EPS = (double)(1e-10);
3 struct punto { // Puede usarse para vectores
4     tipo x, y;
5     punto const operator -(const punto &p) const
        {return {x-p.x,y-p.y};}
6     punto const operator +(const punto &p) const
        {return {x+p.x,y+p.y};}
7     tipo operator *(const punto &p) const {
        return x*p.x + y*p.y;}
8     tipo operator ^(const punto &p) const {
        return x*p.y - y*p.x;}
9     bool operator == (const punto &p) const {
10         return (abs(x-p.x) < EPS && abs(y-p.y) <
            EPS); // Para double
11     }
12     bool operator <(punto p) const {return x != p
        .x ? x < p.x : y < p.y;}
13     tipo arg() {return atan2(y,x);}
14     tipo mod() {return sqrtl(x*x + y*y);}
15     tipo mod2() {return x*x + y*y;}
16 };
17 punto operator*(tipo k, const punto &p) {return
    {k*p.x, k*p.y};}

```

4.9 Minimum Enclosing Circle

```

1 punto getCircleCenter(punto a, punto b) {
2     ld x = a.mod2(), y = b.mod2(), z = a^b;
3     return {(b.y * x - a.y * y) / (2 * z), (a.x *
        y - b.x * x) / (2 * z) };
4 }
5 circ circleFrom(const punto &a, const punto &b,
    const punto &c) {
6     punto o = getCircleCenter(b-a,c-a);
7     return {o+a, o.mod()};
8 }
9 circ spanning_circle(vector<punto> &T) {
10     int n = T.size();
11     shuffle(all(T),rng);
12     circ C={{0,0}, -1};
13     forn(i,n) if (dist(C.c,T[i])>C.r+EPS) {
14         C = {T[i], 0};
15         forn(j,i) if (dist(C.c,T[j])>C.r+EPS) {
16             C = {0.5*(T[i] + T[j]), dist(T[i],T[j]
                )/2};
17             forn(k,j) if (dist(C.c,T[k])>C.r+EPS)
                {
18                 C = circleFrom(T[i], T[j], T[k]);
19             }
20         }
21     }
22     return C;
23 }

```

5 Grafos

5.1 2Sat

```

1 struct two_sat { // 2*x es x, 2*x+1 es ~x
2     int tot;
3     vector<vector<int>> g, g_trans;
4     vb used, assignment;
5     vector<int> order, comp;
6     two_sat(int _tot): tot(_tot){
7         g.resize(tot); g_trans.resize(tot);

```

```

8     } // tot = (normales + negados)
9     void dfs1(int v) {
10         used[v] = true;
11         for(auto u: g[v]) if(!used[u]) dfs1(u);
12         order.pb(v);
13     }
14     void dfs2(int v, int cl) {
15         comp[v] = cl;
16         for(auto u: g_trans[v]) if(comp[u] == -1)
17             dfs2(u, cl);
18     }
19     bool solve() {
20         order.clear(); used.assign(tot, false);
21         comp.assign(tot, -1);
22         forn(i, tot) if(!used[i]) dfs1(i);
23         int comp_act = 0;
24         forn(i, tot){
25             auto v = order[tot-i-1];
26             if(comp[v] == -1) dfs2(v, comp_act++);
27         }
28         assignment.assign(tot/2, false);
29         forn(i, tot/2){
30             if(comp[2*i] == comp[2*i + 1]) return
31                 false;
32             assignment[i] = comp[2*i] > comp[2*i
33                 +1];
34         }
35         return true;
36     }
37     void add_edge(int from, int to){
38         g[from].pb(to); // from implica to
39         g_trans[to].pb(from);
40     }
41     void add_or(int v1, int v2) {
42         add_edge(v1 ^ 1, v2); // ~v1 -> v2
43         add_edge(v2 ^ 1, v1); // ~v2 -> v1
44     } // set x en true/false:
45     // add_or(x, x)/add_or(~x, ~x)

```

5.2 Bellman Ford

```

1 // Time: O(VE)
2 typedef long long tipo;
3 const int MAXN = 3000;
4 tipo INF = (tipo)(1e18+7);
5 struct arista {
6     int x, y; tipo w;
7 };
8 struct nodo {
9     int p; tipo d;
10 }; //f -> parent, d -> distance
11 vector<nodo> ans(MAXN);
12 vector<int> ciclo;
13 bool bFord(vector<arista> &lista, int n, int
14     start) {
15     forn(i,n) ans[i].p = i, ans[i].d = INF;
16     ans[start].d = 0; int x;
17     forn(i,n) {
18         x = -1;
19         for(arista u : lista) {
20             if(ans[u.y].d > ans[u.x].d + u.w) {
21                 ans[u.y].d = ans[u.x].d + u.w;
22                 ans[u.y].p = u.x;
23                 x = u.y;

```

```

23     }
24     }
25     }
26     if(x == -1) return false;
27     else {
28         forn(i,n) x = ans[x].p;
29         for(int v = x ;; v = ans[v].p) {
30             ciclo.push_back(v);
31             if(v == x && ciclo.size() > 1) break;
32         }
33         reverse(all(ciclo));
34         return true;
35     }
36 }

```

5.3 Biconnected

```

1 struct edge{ ll u, v, id; };
2 struct graph{
3     int n;
4     vector<vii> adj; // (to, id)
5     vector<edge> edges;
6     graph(int _n) : n(_n), adj(_n) {}
7     void add_edge(ll u, ll v){
8         ll id = SZ(edges);
9         adj[u].pb({v, id}); adj[v].pb({u, id});
10        edges.pb({u, v, id});
11    }
12    int add_node(){ adj.pb({}); return n++; }
13    vii& operator[] (ll u) { return adj[u]; }
14 };
15 struct BCC{
16     int n; graph adj;
17     vector<vi> comps;
18     vi num, low, art, stk, bridge;
19     BCC(graph &adj): n(adj.n), adj(_adj){
20         num.resize(n), low.resize(n), art.resize(
21             n), bridge.resize(SZ(adj.edges));
22         for (ll u = 0, t; u < n; ++u) if (!num[u
23             ]) dfs(u, -1, t = 0);
24     }
25     void dfs(ll v, ll p, ll &t){
26         num[v] = low[v] = ++t;
27         stk.pb(v);
28         for(auto [u, id]: adj[v]) if (u != p){
29             if (!num[u]){
30                 dfs(u, v, t);
31                 low[v] = min(low[v], low[u]);
32                 if(low[u] > num[v]) bridge[id] = true;
33                 if(low[u] >= num[v]){
34                     art[v] = (num[v] > 1 || num[u
35                         ] > 2);
36                     comps.pb({v});
37                     while (comps.back().back() !=
38                         u)
39                         comps.back().pb(stk.back
40                             ()), stk.pop_back();
41                 }
42                 else low[v] = min(low[v], num[u]);
43             }
44         }
45     }
46     pair<vi, graph> build_tree(){
47         graph tree(0); vi id(n);
48         forn(v, n) if (art[v]) id[v] = tree.

```

```

44         add_node();
45         for (auto &comp : comps){
46             ll node = tree.add_node();
47             for(ll v: comp){
48                 if (!art[v]) id[v] = node;
49                 else tree.add_edge(node, id[v]);
50             }
51         }
52         return {id, tree};
53     } // Block Cut Tree
54 };

```

5.4 Blossom

```

1  /* O(nm) Los matchings estan en (i, mate[i]), con
2     i < mate[i] */
3  vi Blossom(vector<vi> &g) {
4      int n = SZ(g), timer = -1;
5      vi mate(n, -1), label(n), parent(n),
6          orig(n), aux(n, -1), q;
7      auto lca = [&](int x, int y) {
8          for (timer++; ; swap(x, y)) {
9              if (x == -1) continue;
10             if (aux[x] == timer) return x;
11             aux[x] = timer;
12             x = (mate[x] == -1 ? -1 : orig[parent[mate[
13                 x]]]);
14         }
15     };
16     auto blossom = [&](int v, int w, int a) {
17         while (orig[v] != a) {
18             parent[v] = w; w = mate[v];
19             if (label[w] == 1) label[w] = 0, q.pb(w);
20             orig[v] = orig[w] = a; v = parent[w];
21         }
22     };
23     auto augment = [&](int v) {
24         while (v != -1) {
25             int pv = parent[v], nv = mate[pv];
26             mate[v] = pv; mate[pv] = v; v = nv;
27         }
28     };
29     auto bfs = [&](int root) {
30         fill(all(label), -1);
31         iota(all(orig), 0);
32         q.clear();
33         label[root] = 0; q.pb(root);
34         forn(i, SZ(q)){
35             int v = q[i];
36             for (auto x : g[v]) {
37                 if (label[x] == -1) {
38                     label[x] = 1; parent[x] = v;
39                     if (mate[x] == -1)
40                         return augment(x), 1;
41                     label[mate[x]] = 0; q.pb(mate[x]);
42                 } else if (label[x] == 0 && orig[v] !=
43                     orig[x]) {
44                     int a = lca(orig[v], orig[x]);
45                     blossom(x, v, a); blossom(v, x, a);
46                 }
47             }
48         }
49         return 0;
50     }; //Time halves if start with any maxmatch.
51     forn(i, n) if(mate[i] == -1) bfs(i);

```

```

49     return mate;
50 }

```

5.5 Componentes Fuertemente Conexas

```

1  struct SCC {
2      int n, scc;
3      vector<vi> g, gr, ans;
4      vb visto;
5      vi order, comp_act, component;
6      SCC(vector<vi> &_g): n(SZ(_g)) {
7          g = _g;
8          gr.resize(n), visto.resize(n), component.
9              resize(n);
10         forn(v, n) for(auto u: g[v]) gr[u].pb(v);
11         find_scc();
12     }
13     void DFS1 (int v) {
14         visto[v] = true;
15         for (int u : g[v]) if(!visto[u]) DFS1(u);
16         order.pb(v);
17     }
18     void DFS2 (int v) {
19         visto[v] = true;
20         comp_act.pb(v);
21         for (int u : gr[v]) if(!visto[u]) DFS2(u);
22     }
23     void find_scc() {
24         fill(all(visto), false);
25         forn(i, n) if(!visto[i]) DFS1(i);
26         fill(all(visto), false);
27         forn(i, n) {
28             int v=order[n-i-1];
29             if(!visto[v]) {
30                 DFS2(v);
31                 ans.pb(comp_act);
32                 comp_act.clear();
33             }
34         }
35         scc = SZ(ans); // cantidad de scc's
36         forn(i, scc) for(auto v: ans[i])
37             component[v] = i;

```

5.6 Dijkstra

```

1  struct arista {
2      int next; tipo w;
3  };
4  struct nodo {
5      tipo d, v, a;
6      bool operator<(const nodo& next) const {
7          return d > next.d;}
8  };
9  vector<nodo> Dijkstra(int start, int n, vector<
10     vector<arista>> &g) {
11     vector<nodo> ans(n);
12     vector<bool> visto(n, false);
13     priority_queue<nodo> p; p.push({0,start,-1});
14     while(!p.empty()) {
15         nodo it=p.top(); p.pop();
16         if(visto[it.v]) continue;
17         else {
18             ans[it.v] = it; visto[it.v] = true;

```

```

17         for(arista u : g[it.v]) {
18             if(!visto[u.next]) p.push({it.d +
                u.w, u.next, it.v});
19         }
20     }
21 }
22 return ans;
23 }

```

5.7 Dinic

```

1 // Max Flow en  $O(V^2 E)$ . Bipartito  $O(\sqrt{V} E)$ .
2 // Matching: aristas saturadas
3 // Min cut: nodos con dist>=0 vs nodos con dist<0
4 // MVC: izq con dist<0 + der con dist>0
5 // Max Independent Set: comple de MVC (N-MVC)
6 struct Dinic{
7     int nodes,src,dst;
8     vector<int> dist,q,work;
9     struct edge { int to,rev; ll f,cap; };
10    vector<vector<edge>> g;
11    Dinic(int x):nodes(x),dist(x),q(x),work(x),g(
        x){}
12    void add_edge(int s, int t, ll cap){
13        g[s].pb({t,SZ(g[t]),0,cap});
14        g[t].pb({s,SZ(g[s])-1,0,0});
15    }
16    bool dinic_bfs(){
17        fill(all(dist),-1);dist[src]=0;
18        int qt=0;q[qt++]=src;
19        forn(qh, qt){
20            int u=q[qh];
21            forn(i,SZ(g[u])){
22                edge &e=g[u][i];int v=g[u][i].to;
23                if(dist[v]<0&&e.f<e.cap)dist[v]=
                    dist[u]+1,q[qt++]=v;
24            }
25        }
26        return dist[dst]>=0;
27    }
28    ll dinic_dfs(int u, ll f){
29        if(u==dst)return f;
30        for(int &i=work[u];i<SZ(g[u]);i++){
31            edge &e=g[u][i];
32            if(e.cap<=e.f)continue;
33            int v=e.to;
34            if(dist[v]==dist[u]+1){
35                ll df=dinic_dfs(v,min(f,e.cap-e.f
                    ));
36                if(df>0){e.f+=df;g[v][e.rev].f-=
                    df;return df;}
37            }
38        }
39        return 0;
40    }
41    ll max_flow(int _src, int _dst){
42        src=_src;dst=_dst;
43        ll result=0;
44        while(dinic_bfs()){
45            fill(all(work),0);
46            while(ll delta=dinic_dfs(src,INF))
                result+=delta;
47        }
48        return result;
49    }

```

```

50 };
```

5.8 Dynamic Connectivity

```

1 struct UnionFind {
2     int n,comp;
3     vi uf,si,c;
4     UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1)
        {
5         forn(i,n) uf[i]=(int)i;
6     }
7     int find(int x){return x==uf[x]?x:find(uf[x])
        ;}
8     bool join(int x, int y){
9         if((x=find(x))==y)return false;
10        if(si[x]<si[y])swap(x,y);
11        si[x]+=si[y];uf[y]=x;comp--;c.pb(y);
12        return true;
13    }
14    int snap(){return SZ(c);}
15    void rollback(int snap){
16        while(SZ(c)>snap){
17            int x=c.back();c.pop_back();
18            si[uf[x]]-=si[x];uf[x]=x;comp++;
19        }
20    }
21 };
22 enum {ADD,DEL,QUERY};
23 struct Query {int type,x,y};
24 struct DynCon {
25     vector<Query> q;
26     UnionFind dsu;
27     vector<int> mt;
28     map<pair<int,int>,int> last;
29     DynCon(int n):dsu(n){}
30     void add(int x, int y){
31         if(x>y)swap(x,y);
32         q.pb({ADD,x,y});mt.pb(-1);last[{x,y}]=SZ(
            q)-1;
33     }
34     void remove(int x, int y){
35         if(x>y)swap(x,y);
36         q.pb({DEL,x,y});
37         int pr=last[{x,y}];mt[pr]=SZ(q)-1;mt.pb(
            pr);
38     }
39     void query(int x, int y){q.pb({QUERY,x,y});mt
        .pb(-1);}
40     void process(){ // answers all queries in
        order
41         if(!SZ(q)) return;
42         forn(i,SZ(q))if(q[i].type==ADD&&mt[i]<0)
            mt[i]=SZ(q);
43         go(0,SZ(q));
44     }
45     void go(int s, int e){
46         if(s+1==e){
47             if(q[s].type==QUERY) // answer query
                using DSU q[s]
48             assert(false); // alguna op entre
                q[s].x y q[s].y
49             return;
50         }
51         int k=dsu.snap(),m=(s+e)/2;
52         for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]

```

```

    ]<s>dsu.join(q[i].x,q[i].y);
53     go(s,m);dsu.rollback(k);
54     for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.
        join(q[i].x,q[i].y);
55     go(m,e);dsu.rollback(k);
56 }
57 };

```

5.9 Eulerian Cycle

```

1 // Directed version (uncomment commented code for
  undirected)
2 struct edge {
3     int y;
4     // list<edge>::iterator rev;
5     edge(int _y):y(_y){}
6 };
7 list<edge> g[MAXN];
8 void add_edge(int a, int b){
9     g[a].push_front(edge(b));//auto ia=g[a].begin()
10    ;
11    // g[b].push_front(edge(a));auto ib=g[b].begin()
12    ;
13    // ia->rev=ib;ib->rev=ia;
14 }
15 vector<int> p;
16 void go(int x){
17     while(SZ(g[x])){
18         int y=g[x].front().y;
19         //g[y].erase(g[x].front().rev);
20         g[x].pop_front();
21         go(y);
22     }
23     p.pb(x);
24 }
25 vector<int> get_path(int x){ // get a path that
  begins in x
26 // check that a path exists from x before calling
  to get_path!
27 p.clear();go(x);reverse(all(p));
  return p;
}

```

5.10 Floyd Warshall

```

1 void floyd(vector<vector<int>>&matriz, ll n) {
2     forn(k,n) forn(i,n) forn(j,n) {
3         matriz[i][j]=min(matriz[i][j],matriz[i][k]
4         ]+matriz[k][j]);
5     } // O(n^3)
6 }

```

5.11 Fordfulkerson

```

1 struct FordFulkerson {
2     // Time complexity ((V+E) * MAXFLOW)
3     vector < vector<arista> > g;
4     map < pair<int,int>,int > cap;
5     FordFulkerson(int n) {
6         g.resize(n+5);
7     }
8     void add_edge(int x, int y, int z) {
9         g[x].pb({y});
10        g[y].pb({x});
11        cap[{x,y}] = z;
12    } // cap[{x,y}] += z; if multiedges allowed

```

```

13 void DFS(int cur, vector<int> &parent, vector<
  int> &flow, int prev, int cur_flow = INF) {
14     parent[cur] = prev;
15     flow[cur] = cur_flow;
16     for(auto [next] : g[cur]) {
17         if(parent[next] != -1 or cap[{cur,next}] ==
18             0) continue;
19         DFS(next,parent,flow,cur,min(cur_flow,cap[{
20             cur,next}]));
21     }
22 }
23 int max_flow(int s, int t) {
24     int maxflow = 0;
25     vector<int> parent(MAXN,-1), flow(MAXN,-1);
26     DFS(s,parent,flow,s);
27     while(flow[t] > 0) {
28         int new_flow = flow[t];
29         maxflow += new_flow;
30         int cur = t;
31         while(cur != s) {
32             int prev = parent[cur];
33             cap[{prev,cur}] -= new_flow;
34             cap[{cur,prev}] += new_flow;
35             cur = prev;
36         }
37         fill(all(parent),-1);
38         fill(all(flow),0);
39         DFS(s,parent,flow,s);
40     }
41     return maxflow;
}

```

5.12 Hungarian

```

1 // O(n^3) assignment problem
2 typedef long double td; typedef vector<td> vd;
3 const td INF=1e100;//for maximum set INF to 0,
  and negate costs
4 bool zero(td x){return fabs(x)<1e-9;}//change to
  x==0, for ints/ll
5 struct Hungarian{
6     int n; vector<vd> cs; vi L, R;
7     Hungarian(int N, int M):n(max(N,M)),cs(n,vd(n
8     )),L(n),R(n){
9         forn(x,N)forn(y,M)cs[x][y]=INF;
10    }
11    void set(int x,int y,td c){cs[x][y]=c;}
12    td assign() {
13        int mat = 0; vd ds(n), u(n), v(n); vi dad
14        (n), sn(n);
15        forn(i,n)u[i]=*min_element(all(cs[i]));
16        forn(j,n){v[j]=cs[0][j]-u[0];forr(i,1,n)v
17        [j]=min(v[j],cs[i][j]-u[i]);}
18        L=R=vi(n, -1);
19        forn(i,n)forn(j,n)
20            if(R[j]==-1&&zero(cs[i][j]-u[i]-v[j])){L[i]=j;R[j]=i;mat++;break;}
21        for(;mat<n;mat++){
22            int s=0, j=0, i;
23            while(L[s] != -1)s++;
24            fill(all(dad),-1);fill(all(sn),0);
25            forn(k,n)ds[k]=cs[s][k]-u[s]-v[k];
26            for(;;){
27                j = -1;

```

```

25     forn(k,n)if(!sn[k]&&(j== -1 || ds[k]
26         ]<ds[j]))j=k;
27     sn[j] = 1; i = R[j];
28     if(i == -1) break;
29     forn(k,n)if(!sn[k]){
30         auto new_ds=ds[j]+cs[i][k]-u[
31             i]-v[k];
32         if(ds[k] > new_ds){ds[k]=
33             new_ds;dad[k]=j;}
34     }
35     forn(k,n)if(k!=j&&sn[k]){auto w=ds[k]
36         ]-ds[j];v[k]+=w,u[R[k]]-=w;}
37     u[s] += ds[j];
38     while(dad[j]>=0){int d = dad[j];R[j]=
39         R[d];L[R[j]]=j;j=d;}
40     R[j]=s;L[s]=j;
41     td value=0;forn(i,n)value+=cs[i][L[i]];
42     return value;
43 }
44 };

```

5.13 Max Flow Min Cost Multiedge

```

1  typedef long long tipo;
2  typedef long long tipo_cost;
3  const int MAXN = 505;
4  tipo INF = (tipo)(1e9+7);
5  tipo_cost MAX_COST = (tipo_cost)(1e16);
6  struct arista {
7      int v;
8      tipo cap;
9      tipo_cost c; // Cost
10     int id;
11 };
12 struct prev_node {
13     int prev, id;
14     void setting() {prev = -1, id = -1;}
15 };
16 vector < vector<arista> > g(MAXN);
17 vector <tipo> cap;
18 void add_edge(int x, int y, tipo z, tipo_cost c)
19 {
20     int id = cap.size();
21     g[x].pb({y,z,c,id});
22     g[y].pb({x,0,-1*c,id+1});
23     cap.pb(z); cap.pb(0);
24 }
25 void Bellman_Ford(int s, int t, vector <prev_node
26     > &parent, vector <tipo_cost> &d) {
27     for(prev_node u : parent) u.setting();
28     fill(all(d),MAX_COST);
29     vector <bool> inq(MAXN,false);
30     queue <int> q; q.push(s); parent[s] = {s,-1};
31     inq[s] = true; d[s] = 0;
32     while(q.size() > 0) {
33         int u = q.front(); q.pop(); inq[u] = false;
34         for(arista next : g[u]) {
35             if(cap[next.id]>0 && d[next.v] > d[u] +
36                 next.c) {
37                 d[next.v] = d[u] + next.c;
38                 parent[next.v] = {u,next.id};
39                 if(!inq[next.v]) {
40                     inq[next.v] = true;

```

```

37         q.push(next.v);
38     }
39 }
40 }
41 }
42 }
43 tipo_cost max_flow_min_cost(int s, int t, int
44     total) {
45     tipo flow = 0;
46     tipo_cost min_cost = 0;
47     vector <prev_node> parent(MAXN);
48     vector <tipo_cost> d(MAXN);
49     while(true) {
50         Bellman_Ford(s,t,parent,d);
51         if(d[t] == MAX_COST) break;
52         tipo new_flow = INF;
53         int cur = t;
54         while(cur != s) {
55             new_flow = min(new_flow,cap[parent[cur].id
56                 ]);
57             cur = parent[cur].prev;
58         }
59         flow += new_flow;
60         min_cost += d[t] * (tipo_cost)(new_flow);
61         cur = t;
62         while(cur != s) {
63             int prev = parent[cur].prev;
64             int id = parent[cur].id;
65             cap[id] -= new_flow;
66             cap[(id^1)] += new_flow;
67             cur = prev;
68         }
69     }
70     if(flow < total) return -1;
71     return min_cost;

```

5.14 Min Cost Flow

```

1  /* O(n^3 m). Usa Dijkstra multiedge, pero no
2     ciclos neg de costo*/
3  typedef ll tf; // tipo que se usa para el flow
4  typedef ll tc; // tipo que se usa para el cost
5  const tf INFFLOW=1e9;
6  const tc INFCOST=1e9;
7  struct MCF {
8      int n;
9      vector<tc> prio, pot; vector<tf> curflow;
10     vector<int> prevedge,prevnode;
11     priority_queue<pair<tc, int>, vector<pair<tc,
12         int>>, greater<pair<tc, int>>> q;
13     struct edge{int to, rev; tf f, cap; tc cost;};
14     vector<vector<edge>> g;
15     MCF(int _n):n(_n),prio(_n),pot(_n),curflow(_n),
16         prevedge(_n),prevnode(_n),g(_n){}
17     void add_edge(int s, int t, tf cap, tc cost) {
18         g[s].pb({t,SZ(g[t]),0,cap,cost});
19         g[t].pb({s,SZ(g[s])-1,0,0,-cost});
20     }
21     pair<tf,tc> get_flow(int s, int t) {
22         tf flow=0; tc flowcost=0;
23         while(1){
24             q.push({0, s});
25             fill(all(prio),INFCOST);
26             prio[s]=0; curflow[s]=INFFLOW;

```

```

23 while(!q.empty()) {
24     auto cur=q.top(); q.pop();
25     tc d=cur.first; int u=cur.second;
26     if(d!=prio[u]) continue;
27     forn(i, SZ(g[u])){
28         edge &e=g[u][i];
29         int v=e.to;
30         if(e.cap<=e.f) continue;
31         tc nprio=prio[u]+e.cost+pot[u]-pot[v];
32         if(prio[v]>nprio) {
33             prio[v]=nprio;
34             q.push({nprio, v});
35             prevnode[v]=u; prevedge[v]=i;
36             curflow[v]=min(curflow[u], e.cap-e.f)
37         }
38     }
39 }
40 if(prio[t]==INFCOST) break;
41 forn(i, n) pot[i]+=prio[i];
42 tf df=min(curflow[t], INFFLOW-flow);
43 flow+=df;
44 for(int v=t; v!=s; v=prevnode[v]) {
45     edge &e=g[prevnode[v]][prevedge[v]];
46     e.f+=df; g[v][e.rev].f-=df;
47     flowcost+=df*e.cost;
48 }
49 }
50 return {flow,flowcost};
51 }
52 };

```

5.15 Mst Directed

```

1 struct Edge { int a, b; ll w; };
2 struct Node { /// lazy skew heap node
3     Edge key;
4     Node *l, *r;
5     ll delta;
6     void prop() {
7         key.w += delta;
8         if (l) l->delta += delta;
9         if (r) r->delta += delta;
10        delta = 0;
11    }
12    Edge top() { prop(); return key; }
13 };
14 Node *merge(Node *a, Node *b) {
15     if (!a || !b) return a ? b;
16     a->prop(), b->prop();
17     if (a->key.w > b->key.w) swap(a, b);
18     swap(a->l, (a->r = merge(b, a->r)));
19     return a;
20 }
21 void pop(Node& a) { a->prop(); a = merge(a->l, a->r); }
22 pair<ll, vi> dmst(int n, int r, vector<Edge>& g)
23 {
24     UnionFind uf(n);
25     vector<Node*> heap(n);
26     for (Edge e : g) heap[e.b] = merge(heap[e.b],
27         new Node{e});
28     ll res = 0;
29     vi seen(n, -1), path(n), par(n);
30     seen[r] = r;

```

```

29 vector<Edge> Q(n), in(n, {-1,-1}), comp;
30 deque<tuple<int, int, vector<Edge>>> cys;
31 forr(s,0,n) {
32     int u = s, qi = 0, w;
33     while (seen[u] < 0) {
34         if (!heap[u]) return {-1,{};};
35         Edge e = heap[u]->top();
36         heap[u]->delta -= e.w, pop(heap[u]);
37         Q[qi] = e, path[qi++] = u, seen[u] = s;
38         res += e.w, u = uf.find(e.a);
39         if (seen[u] == s) { /// found cycle,
40             contract
41             Node* cyc = 0;
42             int end = qi, time = uf.snap();
43             do cyc = merge(cyc, heap[w = path[--qi]])
44             ;
45             while (uf.join(u, w));
46             u = uf.find(u), heap[u] = cyc, seen[u] =
47                 -1;
48             cys.push_front({u, time, {&Q[qi], &Q[end
49                 ]}});
50         }
51     }
52     forr(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
53 }
54 for (auto& [u,t,comp] : cys) { // restore sol
55     (optional)
56     uf.rollback(t);
57     Edge inEdge = in[u];
58     for (auto& e : comp) in[uf.find(e.b)] = e;
59     in[uf.find(inEdge.b)] = inEdge;
60 }
61 forr(i,0,n) par[i] = in[i].a;
62 return {res, par};
63 }

```

5.16 Toposort

```

1 vi tsort(vector<vi> &g, int n){
2     vi r; priority_queue<ll> q;
3     vi d(2*n);
4     forn(i, n) forn(j: g[i]) d[j]++;
5     forn(i, n) if(!d[i]) q.push(-i);
6     while(!q.empty()){
7         ll x=-q.top(); q.pop(); r.pb(x);
8         forn(j: g[x]){
9             d[j]--;
10            if(!d[j]) q.push(-j);
11        }
12    }
13    return r;
14 } // lexicographically smallest

```

6 Math

Números de Bell (B_n)

Particiones de un conjunto de n elementos. $B_0 = 1$.

Recursiva:

$$B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$$

Números de Catalan (C_n)

Paréntesis, triangulaciones, árboles binarios. $C_0 = 1$.

Explícita:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

6.1 Fft Mod

```
1 typedef double ld;
2 typedef complex<ld> cd;
3 void fft(vector<cd> &a){
4     int n=SZ(a), L=31-__builtin_clz(n);
5     vi rev(n);
6     forn(i, n) rev[i]=(rev[i/2]+((i&1)<<L))/2;
7     forn(i, n) if(i<rev[i]) swap(a[i],a[rev[i]]);
8     static vector<cd> root(2,1);
9     for(static int k=2;k<n;k*=2){
10         root.resize(n);
11         cd z=polar(1.0,acos(-1)/k);
12         forr(i, k, 2*k) root[i]=root[i/2]*((i&1)?
13             z:1);
14     }
15     for(int k=1;k<n;k*=2){
16         for(int i=0;i<n;i+=2*k){
17             forn(j, k){
18                 cd z=root[j+k]*a[i+j+k];
19                 a[i+j+k]=a[i+j]-z;
20                 a[i+j]+=z;
21             }
22         }
23     }
24     vi mul(vi &a, vi &b){
25         int sa=SZ(a), sb=SZ(b);
26         if(sa==0||sb==0) return {};
27         int n=1<<(32-__builtin_clz(sa+sb-2));
28         vector<cd> f(n),h(n);
29         forn(i, n) f[i]=cd(ld(i<sa?a[i]:0),ld(i<sb?b[i]:0));
30         fft(f);
31         forn(i, n){
32             ll j=(-i)&(n-1);
33             h[i]=(f[j]*f[j]-conj(f[i]*f[i]))*cd
34                 (0,-0.25/n);
35         }
36         fft(h);
37         vi c(sa+sb-1);
38         forn(i, sa+sb-1) c[i]=ll(round(h[i].real()));
39         return c;
40     }
41     vi mul_mod(vi &a, vi &b, ll mod){
42         int sa=SZ(a), sb=SZ(b);
43         if(sa==0||sb==0) return {};
44         vi a1(sa),a2(sa),b1(sb),b2(sb);
45         forn(i, sa) a1[i]=(a[i]&((1<<15)-1)),a2[i]=(a[i]>>15);
46         forn(i, sb) b1[i]=(b[i]&((1<<15)-1)),b2[i]=(b[i]>>15);
47         vi c1=mul(a1,b1),c2=mul(a1,b2),c3=mul(a2,b1),
48             c4=mul(a2,b2);
49         forn(i, sa+sb-1) c[i]=(c1[i]+((1<<15)*(c2[i]%
50             mod+c3[i]%mod))+((1<<30)*(c4[i]%mod)))%
51             mod;
52         return c;
53     }
54 }
```

6.2 Fht

```
1 ll c1[MAXN+9],c2[MAXN+9]; // MAXN=pow2
2 void fht(ll* p, int n, bool inv){
3     for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*
4         l)forn(j,l){
5         ll u=p[i+j],v=p[i+l+j];
6         if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
7         else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
8         //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
9         //else p[i+j]=-u+v,p[i+l+j]=u;
10        //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
11        //else p[i+j]=v,p[i+l+j]=u-v;
12    }
13    vi multiply(vi &p1, vi &p2){
14        int n=1<<(32-__builtin_clz(max(SZ(p1),SZ(p2))
15            -1));
16        forn(i,n)c1[i]=0,c2[i]=0;
17        forn(i,SZ(p1))c1[i]=p1[i];
18        forn(i,SZ(p2))c2[i]=p2[i];
19        fht(c1,n,false);fht(c2,n,false);
20        forn(i,n)c1[i]*=c2[i];
21        fht(c1,n,true);
22        return vi(c1,c1+n);
23    }
```

6.3 Gauss Sistema Ecuaciones

```
1 /* Resuelve Ax=B. filas ecuaciones, columnas
2     variables.
3     En ans solucion unica. A es de n x m.
4     n ecuaciones y m-1 variables, ultima columna
5     matriz B.
6     Complejidad O(min(n, m)nm). Si n == m, es O(n^3).
7     */
8 ll mod(ll x) {x%=MOD; if(x<0) x+=MOD; return x;}
9 ll add(ll a, ll b){return mod(a+b);}
10 ll sub(ll a, ll b){return mod(a-b);}
11 ll mul(ll a, ll b){return mod(mod(a)*mod(b));}
12 ll be(ll a, ll p) {
13     ll ans=1; for(; p; p/=2, a = mul(a,a)) if(p&1)
14         ans=mul(ans,a);
15     return ans;
16 }
17 ll divi(ll a, ll b){return mul(a,be(b,MOD-2));}
18 int gauss(vector<vi> &a, vi &ans) {
19     int n = SZ(a), m = SZ(a[0])-1;
20     vector<int> where(m, -1);
21     for(int col=0, row=0; col<m && row<n; ++col)
22     {
23         int sel = row;
24         forr(i, row, n){ // if(abs(a[i][col]) >
25             abs(a[sel][col]))
26             if(a[i][col] > a[sel][col]) sel = int
27                 (i);
28         }
29         if(a[sel][col] == 0) continue; // if(abs(
30             a[sel][col]) < EPS)
31         forr(i, col, m+1) swap(a[sel][i], a[row][
32             i]);
33         where[col] = row;
34         forn(i, n){
35             if(i != row) {
36                 ll c = divi(a[i][col], a[row][col]
37                     )
```

```

    }); // double c = a[i][col] /
    a[row][col];
28     forr(j, col, m+1) a[i][j] = sub(a
        [i][j], mul(a[row][j], c));
29     }
30     }
31     ++row;
32 }
33 ans.assign(m, 0);
34 forn(i, m) if(where[i] != -1) ans[i] = divi(a
    [where[i]][m], a[where[i]][i]);
35 forn(i, n){
36     ll sum = 0; // double sum = 0;
37     forn(j, m) sum = add(sum, mul(ans[j], a[i
        ] [j]));
38     if(sum - a[i][m] != 0) return 0; // if(
        abs(sum-a[i][m]) > EPS)
39     // No hay soluciones
40 }
41 forn(i, m) if(where[i] == -1) return INF;
42 return 1; // Exactamente una solucion
43 }
44 // Encuentra las bases que generan la solucion
45 vector<vi> find_bases(vector<vi> &A, vi &ans) {
46     int n = SZ(A), m = SZ(ans);
47     vi var_ind;
48     forn(i, SZ(ans)) if(ans[i] == 0) var_ind.pb(i);
49     vector<vi> bases(SZ(var_ind), vi(m, 0));
50     int r = 0;
51     for(auto u : var_ind) {
52         bases[r][u] = 1;
53         forn(i, n) {
54             forn(j, m) {
55                 if(A[i][j] != 0) {
56                     bases[r][j] = divi(-A[i][u], A[i][j]);
57                     break;
58                 }
59             }
60         }
61         r++;
62     }
63     return bases;
64 }

```

6.4 Inverse Matrix

```

1 // modular functions in gauss
2 pair<ll, vector<vi>> inverse_matrix(vector<vector<
    ll> > &x) {
3     // returns det and the inverse of the matrix,
4     // if det == 0, no inverse
5     int n = SZ(x); vector<vi> I(n, vi(n, 0));
6     ll det = 1;
7     forn(i, n) I[i][i] = 1;
8     forn(i, n) {
9         int pos = -1;
10        forr(filas, i, n) if(x[filas][i] > 0) pos =
            filas;
11        if(pos == -1) {return {0, I};}
12        if(pos != i) {
13            swap(x[i], x[pos]), swap(I[i], I[pos]);
14            det = mul(det, -1);
15        }
16        ll pivot = x[i][i];
17        det = mul(det, pivot);

```

```

    forn(j, n) {
18        x[i][j] = divi(x[i][j], pivot);
19        I[i][j] = divi(I[i][j], pivot);
20    }
21    }
22    forn(filas, n) {
23        if(filas == i) continue;
24        pivot = x[filas][i];
25        forn(j, n) {
26            x[filas][j] = sub(x[filas][j], mul(pivot, x[i
                ] [j]));
27            I[filas][j] = sub(I[filas][j], mul(pivot, I[i
                ] [j]));
28        }
29    }
30    }
31    return {det, I};
32 }

```

6.5 Lagrange

```

1 ll lagrange(vii a, ll x){
2     ll ans = 0;
3     int n = SZ(a);
4     forn(i, n){
5         ll term = a[i].second;
6         forn(j, n){
7             if(j != i)
8                 term = term*(x-a[j].first)/(a[i].first-a[
                    j].first);
9         }
10        ans += term;
11    }
12    return ans;
13 }

```

6.6 Linear Rec

```

1 const ll LOG = 60;
2 // Linear Recurrence
3 // Description: Calculates the kth term of a
4 // linear recurrence relation
5 // init 0(n^2log) query(n^2 logk)
6 // input: terms: first n term; trans: transition
7 // function (calcular con BM); MOD; LOG=mxlog of
8 // k
9 // output calc(k): kth term mod MOD
10 // example: {1,1} {2,1} an=2*a_(n-1)+a_(n-2);
11 // calc(3)=3 calc(10007)=71480733
12 struct LinearRec {
13     ll n; vi terms, trans; vector<vi> bin;
14     vi add(vi &a, vi &b){
15         vi res(n*2+1);
16         forn(i, n+1) forn(j, n+1) res[i+j]=(res[i
            +j]*1LL+(1ll)a[i]*b[j])%MOD;
17         for(ll i=2*n; i>n; --i){
18             forn(j, n) res[i-1-j]=(res[i-1-j]*1LL
                +(1ll)res[i]*trans[j])%MOD;
19             res[i]=0;
20         }
21         res.erase(res.begin()+n+1, res.end());
22         return res;
23     }
24     LinearRec(vi &terms, vi &trans):terms(terms),
        trans(trans){
25         n=SZ(trans); vi a(n+1); a[1]=1;
26         bin.pb(a);
27     }

```

```

23     forr(i,1,LOG)bin.pb(add(bin[i-1],bin[i
        -1])); // Precompute powers of a
24 }
25 ll calc(ll k){
26     vi a(n+1);a[0]=1;
27     forn(i,LOG) if((k>>i)&1)a=add(a,bin[i]);
28     ll ret=0;
29     forn(i,n) ret=(ret+a[i+1]*terms[i])%MOD;
30     return ret;
31 }
32 };

```

6.7 Matrix Fast Pow

```

1 typedef vector<vector<ll>> Matrix;
2 Matrix ones(int n) {
3     Matrix r(n,vector<ll>(n));
4     forn(i,n)r[i][i]=1;
5     return r;
6 }
7 Matrix operator*(Matrix &a, Matrix &b) {
8     int n=a.size(),m=b[0].size(),z=a[0].size();
9     Matrix r(n,vector<ll>(m));
10    forn(i,n)forn(j,m)forn(k,z)
11        r[i][j]+=a[i][k]*b[k][j],r[i][j]%MOD;
12    return r;
13 }
14 Matrix be(Matrix b, ll e) {
15     Matrix r=ones(b.size());
16     while(e){if(e&1LL)r=r*b;b=b*b;e/=2;}
17     return r;
18 }

```

6.8 Matrix Reduce

```

1 double reduce(vector<vector<double>> &x) {
2     int n=x.size(),m=x[0].size();
3     int i=0,j=0;double r=1.;
4     while(i<n&&j<m){
5         int l=i;
6         forr(k,i+1,n)if(abs(x[k][j])>abs(x[l][j]))l=k;
7         if(abs(x[l][j])<EPS){j++;r=0.;continue;}
8         if(l!=i){r=-r;swap(x[i],x[l]);}
9         r*=x[i][j];
10        for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
11        forr(k,0,n){
12            if(k==i)continue;
13            for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]
                *x[i][l];
14        }
15        i++;j++;
16    } // returns determinant
17    return r;
18 }

```

6.9 Print Int128

```

1 string toString(__int128 num) {
2     string str;
3     do {
4         int digit = num % 10;
5         str = to_string(digit) + str;
6         num = (num - digit) / 10;
7     } while (num != 0);
8     return str;

```

```

9 }
10 ostream& operator<<(std::ostream& os, __int128 t)
11 {
12     string str = toString(t);
13     return os << str;
14 }

```

6.10 Random

```

1 mt19937 rng(chrono::steady_clock::now().
    time_since_epoch().count());
2 struct MyRandom() {
3     uniform_int_distribution<int> dist;
4     MyRandom(int l, int r) : dist(l, r) {}
5     int get(int l, int r) { return dist(rng); }
6 };
7 shuffle(all(v), rng);

```

6.11 Simplex

```

1 struct Simplex {
2     vector<int> X,Y;
3     vector<vector<double>>> A;
4     vector<double> b,c;
5     double z;
6     int n,m;
7     void add_equation(vector<double> v, double
        val){ A.pb(v); b.pb(val); }
8     void set_objective(vector<double> _c){ c = _c;
        }
9     void pivot(int x,int y){
10        swap(X[y],Y[x]);
11        b[x]/=A[x][y];
12        forr(i,0,m)if(i!=y)A[x][i]/=A[x][y];
13        A[x][y]=1/A[x][y];
14        forr(i,0,n)if(i!=x&&abs(A[i][y])>EPS){
15            b[i]-=A[i][y]*b[x];
16            forr(j,0,m)if(j!=y)A[i][j]-=A[i][y]*A
                [x][j];
17            A[i][y]=-A[i][y]*A[x][y];
18        }
19        z+=c[y]*b[x];
20        forr(i,0,m)if(i!=y)c[i]-=c[y]*A[x][i];
21        c[y]=-c[y]*A[x][y];
22    }
23    pair<double,vector<double>> simplex() {
24        // maximiza c^T x, dado Ax<=b, x>=0
25        // retorna par (maximum value, solution
            vector)
26        n=b.size();m=c.size();z=0.;
27        X=vector<int>(m);Y=vector<int>(n);
28        forn(i,m)X[i]=i;
29        forn(i,n)Y[i]=i+m;
30        while(1){
31            int x=-1,y=-1;
32            double mn=-EPS;
33            forn(i,n)if(b[i]<mn)mn=b[i],x=i;
34            if(x<0) break;
35            forr(i,m)if(A[x][i]<-EPS){y=i;break;}
36            assert(y>0 && "No solution found");
37            pivot(x,y);
38        }
39        while(1){
40            double mx=EPS;
41            int x=-1,y=-1;
42            forn(i,m)if(c[i]>mx)mx=c[i],y=i;

```

```

43         if(y<0)break;
44         double mn=1e200;
45         forn(i,n)if(A[i][y]>EPS&&b[i]/A[i][y]
46             ]<mn)mn=b[i]/A[i][y],x=i;
47         assert(x>=0 && "c^T x is unbounded");
48         pivot(x,y);
49     }
50     vector<double> r(m);
51     forn(i,n)if(Y[i]<m)r[Y[i]]=b[i];
52     return mp(z,r);
53 };

```

7 Number Theory

7.1 Binexp Invmod

```

1 ll be(ll b, ll e, ll m = MOD) {
2     ll r=1; b%=m;
3     while(e){if(e&1LL)r=r*b%m;b=b*b%m;e/=2;}
4     return r;
5 }
6 ll inv_mod(ll x, ll m = MOD) {return be(x,m-2,m)
7     ;}

```

7.2 Criba Primos

```

1 vi min_prime;
2 vi criba(ll n) {
3     vb prime(n+1,true);
4     min_prime.resize(n+1,INF);
5     vi primos;
6     for(ll p=2; p*p<=n; p++){
7         if(!prime[p]) continue;
8         for(ll i=p*p; i<=n; i += p) {
9             prime[i] = false;
10            min_prime[i] = min(min_prime[i],p);
11        }
12    }
13    forr(i, 2, n+1) if(prime[i]) primos.pb(i),
14        min_prime[i] = i;
15    return primos;
16 }

```

7.3 Discrete Log

```

1 /* halla x tal que a^x = b (mod m) en O(sqrt(m))
2 */
3 ll discrete_log(ll a, ll b, ll m) {
4     a %= m, b %= m;
5     ll n = sqrt(m) + 1;
6     ll an = 1;
7     for (ll i = 0; i < n; ++i)
8         an = (an * 111 * a) % m;
9     unordered_map<ll, ll> vals;
10    for (ll q = 0, cur = b; q <= n; ++q) {
11        vals[cur] = q;
12        cur = (cur * 111 * a) % m;
13    }
14    for (ll p = 1, cur = 1; p <= n; ++p) {
15        cur = (cur * 111 * an) % m;
16        if (vals.count(cur)) {
17            ll ans = n * p - vals[cur];
18            return ans;
19        }
20    }
21 }

```

```

20     return -1;
21 }

```

7.4 Discrete Root

```

1 // Finds the primitive root modulo p
2 ll generator(ll p) {
3     vi fact;
4     ll phi = p-1, n = phi;
5     for (ll i = 2; i * i <= n; ++i) {
6         if (n % i == 0) {
7             fact.push_back(i);
8             while (n % i == 0)
9                 n /= i;
10        }
11    }
12    if (n > 1)
13        fact.push_back(n);
14    forr(res, 2, p+1){
15        bool ok = true;
16        for (ll factor : fact) {
17            if (be(res, phi / factor, p) == 1) {
18                ok = false;
19                break;
20            }
21        }
22        if (ok) return res;
23    }
24    return -1;
25 }
26 // finds all numbers x such that x^k = a (mod n)
27 vi discrete_root(ll n, ll k, ll a){
28     if (a == 0) return {0};
29     ll g = generator(n);
30     ll sq = (ll) sqrt (n + .0) + 1;
31     vector<pair<ll, ll>> dec(sq);
32     forr(i, 1, sq+1)
33         dec[i-1] = {be(g, i * sq * k % (n - 1), n
34             ), i};
35     sort(all(dec));
36     ll any_ans = -1;
37     forn(i, sq){
38         ll my = be(g, i * k % (n - 1), n) * a % n
39             ;
40         auto it = lower_bound(all(dec), mp(my, 0)
41             );
42         if (it != dec.end() && it->first == my) {
43             any_ans = it->second * sq - i;
44             break;
45         }
46     }
47     if (any_ans == -1) return {};
48     ll delta = (n-1) / __gcd(k, n-1);
49     vi ans;
50     for (ll cur = any_ans % delta; cur < n-1; cur
51         += delta)
52         ans.push_back(be(g, cur, n));
53     sort(ans.begin(), ans.end());
54     return ans;
55 }

```

7.5 Divisors

```

1 vi find_divisors(ll n, vi &primos) {
2     vector <pair<ll,ll>> factor;
3     for(ll prime : primos) {

```

```

4   int cont = 0;
5   while(n % prime == 0) {
6       cont++;
7       n /= prime;
8   }
9   if(cont > 0) factor.pb({prime,cont});
10  }
11  if(n > 1) factor.pb({n,1});
12  vi divisores = {1};
13  for(auto [p,exp] : factor) {
14      int tam = SZ(divisores);
15      forn(i,exp) {
16          forn(j,tam) {
17              int pos = SZ(divisores)-tam;
18              divisores.pb(divisores[pos] * p);
19          }
20      }
21  }
22  sort(all(divisores));
23  return divisores;
24 }

```

7.6 Fast Factorization Mrabin Prho

```

1  typedef long double ld;
2  mt19937_64 rng(chrono::steady_clock::now().
   time_since_epoch().count());
3  ll myrand(ll a, ll b) { return
   uniform_int_distribution<ll>(a, b)(rng); }
4  struct Factorization {
5      inline ll mul(ll a, ll b, ll c){
6          ll s = a * b - c * ll((ld) a / c * b +
7              0.5);
8          return s < 0 ? s + c : s;
9      }
10     ll be(ll a, ll k, ll mod) {
11         ll res = 1;
12         for(; k; k >>= 1, a = mul(a, a, mod)) if
13             (k & 1) res = mul(res, a, mod);
14         return res;
15     }
16     bool miller(ll n) {
17         auto test = [&](ll n, ll a) {
18             if (n == a) return true;
19             if (n % 2 == 0) return false;
20             ll d = (n - 1) >> __builtin_ctzll(n -
21                 1);
22             ll r = be(a, d, n);
23             while (d < n - 1 && r != 1 && r != n
24                 - 1) {
25                 d <<= 1;
26                 r = mul(r, r, n);
27             }
28             return r == n - 1 || d & 1;
29         };
30         if(n == 2) return 1;
31         for(auto p: vi{2, 3, 5, 7, 11, 13}) if(!
32             test(n, p)) return 0;
33         return 1;
34     }
35     ll pollard(ll n) {
36         auto f = [&](ll x) { return ll((__int128(
37             x) * x + 1) % n); };
38         ll x = 0, y = 0, t = 30, prd = 2;
39         while (t++ % 40 || __gcd(prd, n) == 1) {

```

```

34         if (x == y) x = myrand(2, n - 1), y =
35             f(x);
36         ll tmp = mul(prd, abs(x - y), n);
37         if (tmp) prd = tmp;
38         x = f(x), y = f(f(y));
39     }
40     return __gcd(prd, n);
41 }
42 vi factorize(ll n) {
43     vi res;
44     auto dfs = [&](auto &dfs, ll x) {
45         if (x == 1) return;
46         if (miller(x)) res.pb(x);
47         else {
48             ll d = pollard(x);
49             dfs(dfs, d); dfs(dfs, x / d);
50         }
51     };
52     dfs(dfs, n); sort(all(res));
53     return res;
54 }

```

7.7 Teo Chino Resto

```

1  ii extendedEuclid(ll a, ll b){
2      ll x,y; //a * x + b * y = gcd(a,b)
3      if (b==0) return {1, 0};
4      auto p = extendedEuclid(b, a%b);
5      x = p.second, y = p.first-(a/b)*x;
6      if(a*x+b*y == -__gcd(a,b)) x=-x, y=-y;
7      return {x, y};
8  }
9  pair<ii, ii> dioph(ll a, ll b, ll r) {
10     ll d=__gcd(a,b); // a*x+b*y=r
11     a/=d; b/=d; r/=d;
12     auto p = extendedEuclid(a,b);
13     p.first*=r; p.second*=r;
14     assert(a*p.first+b*p.second==r);
15     return {p, {-b,a}}; // sol=p+t*ans.ss
16 }
17 ll inv(ll a, ll mod) { // inv a mod m
18     assert(__gcd(a,mod)==1);
19     ii sol = extendedEuclid(a,mod);
20     return ((sol.first%mod)+mod)%mod;
21 }
22 ii sol(tuple<ll,ll,ll> c){ //requires inv,
   diophantine
23     auto [a, x1, m] = c; ll d = __gcd(a,m);
24     if(d==1) return {mod(x1*inv(a,m),m), m};
25     return x1%d ? mp(-1LL,-1LL) : sol({a/d, x1/d,
26         m/d});
27 }
28 // cond[i] = {ai,bi,mi} ai*xi=bi (mi); assumes
   lcm fits in ll
29 // Mucho cuidado con el overflow, usar __int128
   si lcm no entra en ll
30 pair<ll,ll> crt(vector<tuple<ll,ll,ll>> cond) {
31     ll x1=0,m1=1,x2,m2;
32     for(auto t:cond){
33         tie(x2,m2) = sol(t);
34         if((x1-x2)%__gcd(m1,m2)) return {-1,-1};
35         if(m1==m2) continue;
36         ll k=dioph(m2,-m1,x1-x2).first.second, l=
37             m1*(m2/__gcd(m1,m2));

```

```

36     x1 = mod((__int128)m1*k+x1,1); m1=1;
37 } // returns: (sol, lcm)
38 return sol({1,x1,m1});
39 }

```

8 Python

8.1 Numbs.Py

```

1 import random
2 x = random.randint(1,10) # (including 1 and 10)
3 choices = ["apple", "banana", "cherry"]
4 x = random.choice(choices) # Returns random
   element from list
5 x = 5 % -2 # -1 (because 5 - (-3 * -2) = 5 - 6
   = -1)
6 x = -5 % 2 # 1 (because -5 - (-3 * 2) = -5 -
   (-6) = 1)
7 l = list() # Defining empty list (l = [])
8 l.extend([2,3]) # Extending an existing list
9 l.insert(1, 100) # Inserting at specific index
10 x = l.pop()
11 x = l.pop(2) # Removing from specific index
12 x = l + [4,5] # Merging two lists (does not
   change existing list)
13 l.remove(2) # Removing first occurrence of an
   element
14 l = [0,1,2,3,4,5,6]
15 l1 = l[2:4] # Getting sublist, returns [2,3]
16 l1 = l[:] # Quickly make a copy of list
17 l[0], l[2] = l[2], l[0] # Swapping two elements
18 l = list({"a"}) # Initializing, from set -> ["a"]
19 d = {"a": 1, "c": 2}
20 l = list(d) # From dict keys -> ["a", "c"]
21 l = list(d.keys()) # From dict keys -> ["a", "c"]
22 l = list(d.values()) # From dict values -> [1,2]
23 l = list(d.items()) # From dict key-values -> [("a", 1), ("c", 2)]
24 l = [1]*n # List with 1 repeated n times
25 l = list(range(10)) # Returns [0,1,2,3...,8,9]
26 l = [x+1 for x in nums] # Preparing list from
   another iterable
27 nums = [5,1,4,9,8]
28 for i, n in enumerate(nums): # Iterating with
   index
29     print(f"Index: {i}, number: {n}")
30 for i in range(0,n,2): # Iterating over range
   with step size = 2
31     print(i)
32 for i in range(len(nums)): # Option 1
33     print(nums[i])
34 for i in range(len(nums), -1, -1): # Option 2
35     print(nums[i])
36 for i in range(len(nums)//2): # Two pointer (left
   and right end) iteration
37     print(i) # 0,1,2
38     print(~i) # -1 (i.e. 4), -2 (i.e. 3), -3(i.e.
   2)
39 for x, y in zip(l1, l2): # Iterate over two lists
40     print(x)
41     print(y)
42 for i, (x, y) in enumerate(zip(l1,l2)):
43     print(f"Index: {i}, x: {x}, y: {y}")
44 nums = [1,2,3,4]

```

```

45 for n in nums[:]: # Safely removing element from
   list while iterating
46     if n % 2 == 0:
47         nums.remove(n)
48 nums = [5,1,4,9,8]
49 nums.sort() # In-place sorting
50 nums.sort(reverse=True) # Descending order
51 new_nums = sorted(nums) # Getting new sorted list
52 new_nums = sorted(nums, reverse=True) #
   Descending order
53 nums = [1,-1,3,2,-3,-4]
54 nums.sort(key=abs) # Sort based on absolute value
55 nums = ["apple", "banana", "cherry"]
56 nums.sort(key=len) # Sort based on length
57 nums = [(0,1), (3,1), (1,2)]
58 nums.sort(key = lambda x : x[1]) # Sorts based on
   first element
59 nums = [{"age": 18, "name": "x"}, {"age": 12, "
   name": "y"}]
60 nums.sort(key = lambda x: x["age"]) # Sorts based
   age
61 from collections import Counter
62 c = Counter([1,2,4,1,2,5]) # Counter({1: 2, 2: 2,
   4: 1, 5: 1})
63 for k, v in c.items():
64     print(f"Element: {k}, Frequency: {v}")
65 x = 1 in c # Membership checking
66 c[1] = 1 # Updating frequency
67 del c[1] # Removing character
68 from bisect import bisect_left, bisect_right #
   Equivalent to lower_bound and upper_bound
69 l = [1,2,2,3,4]
70 x = bisect_left(l, -1) # Returns 0
71 x = bisect_right(l, -1) # Returns 0
72 x = bisect_left(l, 10) # Returns 5
73 x = bisect_right(l, 10) # Returns 5
74 x = bisect_left(l, 2) # Returns 1 (index of first
   occurrence of 2)
75 x = bisect_right(l, 2) # Returns 3 (1 + index of
   last occurrence of 2)
76 output = all(x%2 == 0 for x in nums) # Check if
   list has all even numbers
77 output = any(x == 0 for x in nums) # Check if
   list has one or more zero value
78 is_pal = all(s[i] == s[~i] for i in range(len(s)
   //2)) # Check if string is palindrome
79 from itertools import permutations, combinations
80 items = ["A", "B", "C"]
81 for c in combinations(items, 2):
82     print(c) # Prints ("A", "B"), ("A", "C"), ("B",
   "C")
83 for p in permutations(items): # Iterate over all
   permutations
84     print(p)
85 for p in permutations(items, 2): # Iterate over
   all permutations of length 2
86     print(p)
87 d = dict() # Defining empty dict, also d = {}
88 d["a"] = 1 # Add/Update value against key
89 x = d["a"] # Raises KeyError if key is not
   present
90 x = d.get("a", None) # Returns default value if
   key is not present
91 if "a" in d: # Get a value and delete key
92     x = d.pop("a")

```

```

93 if "a" in d: # Delete key
94     del d["a"]
95 sorted_keys = sorted({"x": 1, "a": 2}) # Returns
96     ["a", "x"]
97 d.pop(next(iter(d))) # Pop first inserted key
98 from collections import defaultdict
99 d = defaultdict(int) # initializes non-present
100 key with value 0
101 d = defaultdict(lambda : 1) # initializes non-
102 present key with value 1
103 d = defaultdict(list) # initializes non-present
104 key with empty list ([])
105 d = defaultdict(dict) # initializes non-present
106 key with empty dict ({}))
107 d = { x : x%2 == 0 for x in nums } # Dictionary
108 comprehension : Preparing dict from another
109 iterable
110 d = dict(['()', '[]', '{}']) # Returns {'(:': ')',
111 '[:': ']', '{': '}' } # Create a dictionary
112 from list/tuple of 2 length strings
113 t = (1, 2, 3) # Initialize tuple
114 t = (1,) # Make sure to add a comma to initialize
115 single element tuple correctly # Otherwise
116 it is treated as integer
117 t[0] = 4 # As tuples are immutable, this will
118 raise a TypeError
119 d = {(1, 2): 'value'} # Tuple as dict keys
120 s = {(1, 2), (3, 4)} # Tuple as set elements
121 def multi_value_func(): # Unpacking
122     return 1, 2, 3
123 x, y, z = multi_value_func()
124 s = set() # Defining an empty set
125 s.add(1) # Adding element
126 size = len(s) # Getting size of set
127 s.update([1,2,3]) # Adding multiple elements to
128 set
129 if 1 in s: # Removing already added element
130     s.remove(1)
131 else:
132     print("Removing an element that does not exist
133         in set, throws KeyError.")
134 s.discard(1) # Removing element irrespective of
135 whether it exists in set or not
136 if len(s) == 0: # Checking whether set is empty
137     print("Set is empty")
138 if not s:
139     print("Set is either empty or s is None")
140 s = set([1,2])
141 for i, x in enumerate(s): # Iterating over set
142     print(f"Insertion_Index_{i}:Element_{x}")
143 s1 = {1,2,3}, s2 = {3,4,5}
144 union_set = s1 | s2 # {1,2,3,4,5}
145 intersection_set = s1 & s2 # {3}
146 diff_set = s1 - s2 # {1,2}
147 sym_diff_set = s1 ^ s2 # {1,2,4,5} (Just like XOR
148 ,returns what is not present in both)
149 s1 <= s2 # Subset test (True)
150 s1 >= s2 # Superset test (False)
151 stack = [] # In python, List can be used as Stack
152
153 stack.append(1) # Push
154 x = stack[-1] # Top / Peek
155 x = stack.pop() # Pop
156 size = len(stack) # Getting size of stack
157 from collections import deque

```

```

141 dq = deque() # Defining an empty deque
142 dq.append(1) # Appending element at rear/back
143 dq.appendleft(1) # Appending element at front/
144 head
145 x = dq.pop() # Removing and getting element from
146 rear/back
147 x = dq.popleft() # Removing and getting element
148 from front/head
149 size = len(dq) # Getting length/size of deque
150 x = dq[-1] # Accessing element at rear
151 x = dq[0] # Accessing element at front
152 x = dq[3] # Accessing element at any index
153 if len(dq) == 0: # Checking whether deque is
154     empty
155     print("Deque is empty")
156 if 1 in dq: # Looking for an element
157     print("1 is part of deque")
158 for i, x in enumerate(dq): # Iterating over deque
159     print(f"Index_{i}:Element_{x}")
160 l1 = sorted(dq) # Sorted operation returns new
161 list which is sorted

```

8.2 Strings.Py

```

1 s1s = s.lstrip() # Removes space from left end
2 srs = s.rstrip() # Removes space from right end
3 ss = s.strip() # Removes space from both ends but
4 not from the middle
5 x = "".join(["I", "am", "learning", "python"]) #
6 Returns "Iamlearningpython"
7 x = "\n".join(["I", "am", "learning", "python"]) #
8 With delimiter # Returns "I am learning
9 python"
10 lower_case_s = s.lower()
11 upper_case_s = s.upper()
12 x = s.isalnum() # To check if string has all
13 characters as either digit or alphabet
14 x = s.isalpha() # To check if string has all
15 alphabets
16 x = s.isnum() # To check if string has all digits
17 x = isspace() # To check if string has all spaces
18 x = ord("b") # Returns 98 # Get unicode/ascii
19 representation of character
20 x = s[2:4] # Return "sa" # Slicing
21 x = sorted(s) # Returns ['a', 's', 't', 'u', 'v']
22 x = sorted(s, reverse=True) # Returns ['v', 'u',
23 't', 's', 'a']
24 "a" in "utsav" # Returns True
25 s = "prefix"
26 x = s.startswith("pre") #Returns True
27 x = s.endswith("fix") #Returns True
28 s1 = "hello_world_programming"
29 subs1 = "world"
30 start_index = s1.find(subs1) # Returns 6

```

9 Strings

9.1 Aho Corasick

```

1 struct vertex {
2     map<char,int> next, go;
3     int p,link,nextleaf;
4     char pch;
5     vector<int> leaf;

```

```

6     vertex(int p=-1, char pch=-1,int nextleaf=-1)
7         :
          p(p),pch(pch),link(-1),nextleaf(nextleaf)
          {}
8 };
9 vector<vertex> t;
10 vector <vector <int> > g(MAXN); // Suffix-links
      tree
11 void aho_init(){ //do not forget!!
12     t.clear(); t.pb(vertex());
13 }
14 void add_string(string s, int id) {
15     int v=0;
16     for(char c:s) {
17         if(!t[v].next.count(c)){
18             t[v].next[c]=t.size();
19             t.pb(vertex(v,c));
20         }
21         v=t[v].next[c];
22     }
23     t[v].leaf.pb(id);
24 }
25 int go(int v, char c);
26 int get_link(int v) {
27     if(t[v].link < 0) {
28         if(v == 0 || t[v].p == 0) t[v].link = 0;
29         else t[v].link = go(get_link(t[v].p),t[v]
30                               ].pch);
31         g[t[v].link].pb(v);
32     }
33     return t[v].link;
34 }
35 int go(int v, char c){
36     if(!t[v].go.count(c)) {
37         if(t[v].next.count(c))t[v].go[c]=t[v].
38             next[c];
39         else t[v].go[c] = v == 0 ? 0 : go(
40             get_link(v),c);
41     }
42     return t[v].go[c];
43 }
44 int init_next_leaf(int v) {
45     if(v == 0) t[v].nextleaf=0;
46     if(t[get_link(v)].leaf.size()) return t[v].
47         nextleaf=get_link(v);
48     else return t[v].nextleaf = t[get_link(v)].
49         nextleaf != -1 ?
50         t[get_link(v)].nextleaf :
51         init_next_leaf(get_link(v));
52 }
53 void construct_links() { forn(i,t.size())
54     get_link(i); }

```

9.2 Dynamic Hash

```

1 struct BIT {
2     vector <ll> prefix, a; ll M;
3     BIT() {}
4     BIT(vector <ll> &v, ll MOD) {
5         int n = v.size(); prefix.resize(n+1); a = v;
6         M = MOD;
7         vector <ll> aux(n+1,0);
8         forn(i,n) aux[i+1] = (aux[i] + v[i]) % M;
9         forr(i,1,n+1) prefix[i] = (aux[i] + M - aux[i]
10             - (i&(-i))) % M;

```

```

9     }
10 ll query(int l, int r) { //[a,b] 0-indexed
11     ll ans = 0; r++;
12     while(r) ans += prefix[r], r -= r&(-r), ans
13         %= M;
14     while(l) ans += M - prefix[l], l -= l&(-l),
15         ans %= M;
16     return ans;
17 }
18 void update(int pos, ll val) {
19     int i = pos + 1; ll upd = (val + M - a[pos])
20         % M;
21     while(i < prefix.size()) prefix[i] += upd,
22         prefix[i] %= M, i += i&(-i);
23     a[pos] = val;
24 }
25 };
26 struct Hashing {
27     int n;
28     const ll MOD[2] = {999727999, 1070777777};
29     vector <ll> prefix[2], rev[2], pot[2], inv_pot
30         [2];
31     ll P = 31, invP[2] = {inv_mod(P,MOD[0]),
32         inv_mod(P,MOD[1])};
33     BIT s[2], rs[2];
34     Hashing() {}
35     Hashing(string &pal) {
36         n = pal.size();
37         forn(k,2) {
38             prefix[k].resize(n,0); rev[k].resize(n,0);
39             pot[k].resize(n,1); inv_pot[k].resize(n,1);
40             forn(i,n) {
41                 if(i) pot[k][i] = (pot[k][i-1] * P) % MOD
42                     [k];
43                 if(i) inv_pot[k][i] = (inv_pot[k][i-1] *
44                     invP[k]) % MOD[k];
45                 prefix[k][i] = (ll)(pal[i]-'a') * pot[k][
46                     i] % MOD[k];
47                 rev[k][i] = (ll)(pal[n-i-1]-'a') * pot[k]
48                     [i] % MOD[k];
49             }
50             s[k] = BIT(prefix[k],MOD[k]);
51             rs[k] = BIT(rev[k],MOD[k]);
52         }
53     }
54     pair<ll,ll> get(int l, int r) { //[l,r] 0-
55         indexed
56         ll x = (s[0].query(l,r) * inv_pot[0][l]) %
57             MOD[0];
58         ll y = (s[1].query(l,r) * inv_pot[1][l]) %
59             MOD[1];
60         return {x,y};
61     }
62     pair<ll,ll> getr(int l, int r) { //[l,r] 0-
63         indexed
64         l = n-1-l, r = n-1-r; swap(l,r);
65         ll x = (rs[0].query(l,r) * inv_pot[0][l]) %
66             MOD[0];
67         ll y = (rs[1].query(l,r) * inv_pot[1][l]) %
68             MOD[1];
69         return {x,y};
70     }
71     void update(int pos, char a) {
72         ll val = (ll)(a-'a');
73         forn(k,2) {

```

```

58     s[k].update(pos,(val * pot[k][pos]) % MOD[k]
59     ]);
60     assert(n-1-pos >= 0);
61     rs[k].update(n-1-pos,(val * pot[k][n-1-pos
62     ]) % MOD[k]);
63 }
};

```

9.3 Dynamic Hash.Cpp.Bak

```

1  ll be(ll x, ll y, ll m) {
2      if (y == 0) return 1;
3      ll p = be(x, y/2, m) % m;
4      p = (p * p) % m;
5      return (y%2 == 0)? p : (x * p) % m;
6  }
7  ll inv_mod(ll x, ll m) {return be(x,m-2,m);}
8  struct BIT {
9      vector<ll> prefix, a; ll M;
10     BIT() {}
11     BIT(vector<ll> &v, ll MOD) {
12         int n = v.size(); prefix.resize(n+1); a = v;
13         M = MOD;
14         vector<ll> aux(n+1,0);
15         forn(i,n) aux[i+1] = (aux[i] + v[i]) % M;
16         forr(i,1,n+1) prefix[i] = (aux[i] + M - aux[i
17             - (i&(-i))]) % M;
18     }
19     ll query(int l, int r) { //[a,b] 0-indexed
20         ll ans = 0; r++;
21         while(r) ans += prefix[r], r -= r&(-r), ans
22             %= M;
23         while(l) ans += M - prefix[l], l -= l&(-l),
24             ans %= M;
25         return ans;
26     }
27     void update(int pos, ll val) {
28         int i = pos + 1; ll upd = (val + M - a[pos])
29             % M;
30         while(i < prefix.size()) prefix[i] += upd,
31             prefix[i] %= M, i += i&(-i);
32         a[pos] = val;
33     }
34 };
35 struct Hashing {
36     int n;
37     const ll MOD[2] = {999727999, 1070777777};
38     vector<ll> prefix[2], rev[2], pot[2], inv_pot
39         [2];
40     ll P = 31, invP[2] = {inv_mod(P,MOD[0]),
41         inv_mod(P,MOD[1])};
42     BIT s[2], rs[2];
43     Hashing() {}
44     Hashing(string &pal) {
45         n = pal.size();
46         forn(k,2) {
47             prefix[k].resize(n,0); rev[k].resize(n,0);
48             pot[k].resize(n,1); inv_pot[k].resize(n,1);
49             forn(i,n) {
50                 if(i) pot[k][i] = (pot[k][i-1] * P) % MOD
51                     [k];
52                 if(i) inv_pot[k][i] = (inv_pot[k][i-1] *
53                     invP[k]) % MOD[k];
54                 prefix[k][i] = (ll)(pal[i]-'a') * pot[k][

```

```

        i] % MOD[k];
45         rev[k][i] = (ll)(pal[n-i-1]-'a') * pot[k]
46             [i] % MOD[k];
47     }
48     s[k] = BIT(prefix[k],MOD[k]);
49     rs[k] = BIT(rev[k],MOD[k]);
50 }
51 pair<ll,ll> get(int l, int r) { //[l,r] 0-
52     indexed
53     ll x = (s[0].query(l,r) * inv_pot[0][l]) %
54         MOD[0];
55     ll y = (s[1].query(l,r) * inv_pot[1][l]) %
56         MOD[1];
57     return {x,y};
58 }
59 pair<ll,ll> getr(int l, int r) { //[l,r] 0-
60     indexed
61     l = n-1-l, r = n-1-r; swap(l,r);
62     ll x = (rs[0].query(l,r) * inv_pot[0][l]) %
63         MOD[0];
64     ll y = (rs[1].query(l,r) * inv_pot[1][l]) %
65         MOD[1];
66     return {x,y};
67 }
68 void update(int pos, char a) {
69     ll val = (ll)(a-'a');
70     forn(k,2) {
71         s[k].update(pos,(val * pot[k][pos]) % MOD[k]
72             [i]);
73         assert(n-1-pos >= 0);
74         rs[k].update(n-1-pos,(val * pot[k][n-1-pos
75             ]) % MOD[k]);
76     }
77 }
};

```

9.4 Hashing

```

1  struct Hash {
2      const ll P=17777771;
3      const ll MOD[2] = {999727999, 1070777777};
4      const ll PI[2] = {325255434, 10018302};
5      vector<ll> h[2],pi[2];
6      Hash(string& s){
7          forn(k,2)h[k].resize(s.size()+1),pi[k].
8              resize(s.size()+1);
9          forn(k,2){
10             h[k][0]=0;pi[k][0]=1;
11             ll p=1;
12             forr(i,1,s.size()+1){
13                 h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[
14                     k];
15                 pi[k][i]=(1LL*pi[k][i-1]*PI[k])%
16                     MOD[k];
17                 p=(p*P)%MOD[k];
18             }
19         }
20     }
21     ll get(ll s, ll e){ //[s, e] (s y e van de 0
22         a n-1)
23         e++;
24         ll h0=(h[0][e]-h[0][s]+MOD[0]);
25         h0=(1LL*h0*pi[0][s])%MOD[0];
26         ll h1=(h[1][e]-h[1][s]+MOD[1]);

```

```

23     h1=(1LL*h1*pi[1][s])%MOD[1];
24     return (h0<<32)|h1;
25 }
26 };

```

9.5 Manacher

```

1 vi d1; // impar
2 vi d2; // par
3 //s aabbaacaabbaa
4 //d1 1111117111111
5 //d2 0103010010301
6 void manacher(string& s){
7     ll l=0,r=-1,n=SZ(s);
8     d1.resize(n), d2.resize(n);
9     forn(i, n){
10         ll k = (i>r ? 1 : min(d1[l+r-i],r-i));
11         while(i+k<n && i-k>=0 && s[i+k]==s[i-k])
12             k++;
13         d1[i] = k--;
14         if(i+k>r) l=i-k,r=i+k;
15     }
16     l=0; r=-1;
17     forn(i, n){
18         ll k = (i>r ? 0 : min(d2[l+r-i+1],r-i+1));
19         k++;
20         while(i+k<=n && i-k>=0 && s[i+k-1]==s[i-k-1]) k++;
21         d2[i] = --k;
22         if(i+k-1>r) l=i-k, r=i+k-1;
23     }
24 }

```

9.6 Prefix Function

```

1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     forr(i,1,n) {
5         int j = pi[i-1];
6         while(j > 0 && s[i] != s[j]) j = pi[j-1];
7         if(s[i] == s[j]) j++;
8         pi[i] = j;
9     }
10     return pi;
11 }

```

9.7 Suffix Array

```

1 void csort(vector<int>& sa, vector<int>& r, int k
2 ){
3     int n=sa.size();
4     vector<int> f(max(255,n),0),t(n);
5     forr(i,0,n)f[RB(i+k)]++;
6     int sum=0;
7     forr(i,0,max(255,n))f[i]=(sum+=f[i])-f[i];
8     forr(i,0,n)t[f[RB(sa[i]+k)]++]=sa[i];
9     sa=t;
10 }
11 vector<int> suffix_array(string& s){ // O(n logn)
12     s += '$';
13     int n=s.size(),rank;
14     vector<int> sa(n),r(n),t(n);
15     forr(i,0,n)sa[i]=i,r[i]=s[i];
16     for(int k=1;k<n;k*=2){
17         csort(sa,r,k);csort(sa,r,0);

```

```

17     t[sa[0]]=rank=0;
18     forr(i,1,n){
19         if(r[sa[i]]!=r[sa[i-1]]||RB(sa[i]+k)
20             !=RB(sa[i-1]+k))rank++;
21         t[sa[i]]=rank;
22     }
23     r=t;
24     if(r[sa[n-1]]==n-1)break;
25 }
26 return sa;
27 }
28 vector<int> lcp_construction(string const& s,
29     vector<int> const& p) {
30     int n = s.size();
31     vector<int> rank(n, 0);
32     for (int i = 0; i < n; i++) rank[p[i]] = i;
33     int k = 0;
34     vector<int> lcp(n-1, 0);
35     for (int i = 0; i < n; i++) {
36         if(rank[i] == n - 1) {
37             k = 0;
38             continue;
39         }
40         int j = p[rank[i] + 1];
41         while (i + k < n && j + k < n && s[i+k]
42             == s[j+k]) k++;
43         lcp[rank[i]] = k;
44         if(k) k--;
45     }
46     return lcp;
47 }
48 bool substr_search(string &text, string &pal,
49     vector<int> &sa) {
50     int n = text.size(), a = -1, b = n-1;
51     while(b-a > 1) {
52         int med = (a+b)/2, pos = sa[med]; bool d
53             = false;
54         int tam = min((int)pal.size(),int(n-sa[
55             med]));
56         string check = text.substr(sa[med],tam);
57         if(check < pal) a=med;
58         else b = med;
59     }
60     int tam = min(int(pal.size()),int(n-sa[b]));
61     string fin = text.substr(sa[b],tam);
62     return fin == pal ? true : false;
63 }
64 ll count_substring(string &pal) {
65     ll n = pal.size();
66     vector<int> sa = suffix_array(pal);
67     vector<int> lcp = lcp_construction(pal,sa);
68     ll ans = n * (n+1) / 2;
69     for(int u : lcp) ans -= (ll)u;
70     return ans;
71 }
72 string lcsbstr(string s, string t) {
73     string r = s + '#' + t + '$';
74     int best = 0; int pos = 0;
75     vector<int> sa = suffix_array(r);
76     vector<int> lcp = lcp_construction(r, sa);
77     forr(i, 1, SZ(sa)) {
78         if(isInRange(sa[i - 1], 0, SZ(s)) &&
79             isInRange(sa[i], SZ(s) + 1, SZ(r) -
80                 1)) {
81             if(lcp[i-1] > best) best = lcp[i-1],

```

```

        pos = sa[i];
74     }
75     if(isInRange(sa[i], 0, SZ(s)) &&
        isInRange(sa[i - 1], SZ(s) + 1, SZ(r)
        - 1)) {
76         if(lcp[i-1] > best) best = lcp[i-1],
            pos = sa[i];
77     }
78 }
79 return r.substr(pos,best);
80 }

```

9.8 Suffix Automaton

```

1 struct state {
2     int len, link;
3     map<char,int> next;
4 };
5 const int MAXN = 300010;
6 state st[MAXN*2];
7 int sz, last;
8 void sa_init(){
9     forn(i,2*MAXN) {
10         st[i].next.clear();
11         st[i].link = 0;
12         st[i].len = 0;
13     }
14     last= st[0].len=0; sz=1;
15     st[0].link=-1;
16 }
17 void sa_extend(char c) {
18     int cur = sz++;
19     st[cur].len = st[last].len + 1;
20     int p;
21     for(p = last; p != -1 && !st[p].next.count(c)
        ; p = st[p].link) {
22         st[p].next[c] = cur;
23     }
24     if (p == -1) st[cur].link = 0;
25     else {
26         int q = st[p].next[c];
27         if(st[p].len + 1 == st[q].len) st[cur].
            link = q;
28         else {
29             int clone = sz++;
30             st[clone].len = st[p].len + 1;
31             st[clone].next = st[q].next;
32             st[clone].link = st[q].link;
33             while (p != -1 && st[p].next[c] == q)
34                 st[p].next[c] = clone;
35             p = st[p].link;
36         }
37         st[q].link = st[cur].link = clone;
38     }
39 }
40 last = cur;
41 }

```

9.9 Z Function

```

1 vector<int> z_function(string s) {
2     int n = (int) s.size();
3     vector<int> z(n);
4     for (int i = 1, l = 0, r = 0; i < n; ++i) {
5         if (i <= r)

```

```

        z[i] = min (r - i + 1, z[i - 1]);
7         while (i + z[i] < n && s[z[i]] == s[i + z
            [i]])
8             z[i]++;
9         if (i + z[i] - 1 > r)
10             l = i, r = i + z[i] - 1;
11     }
12     return z;
13 }

```

10 Tree

10.1 Centroid Cses

```

1 vector<int> g[MAXN];int n;
2 bool tk[MAXN];
3 int fat[MAXN]; // father in centroid
4 // decomposition
5 int szt[MAXN]; // size of subtree
6 int calcsz(int x, int f){
7     szt[x]=1;
8     for(auto y:g[x])if(y!=f&&!tk[y])szt[x]+=
        calcsz(y,x);
9     return szt[x];
10 }
11 void cdfs(int x=0, int f=-1, int sz=-1){ // 0(
    // nlogn)
12     if(sz<0)sz=calcsz(x,-1);
13     for(auto y:g[x])if(!tk[y]&&szt[y]*2>=sz){
14         szt[x]=0;cdfs(y,f,sz);return;
15     }
16     tk[x]=true;fat[x]=f;
17     for(auto y:g[x])if(!tk[y])cdfs(y,x);
18 }
19 void centroid(){memset(tk,false,sizeof(tk));cdfs
    ();}

```

10.2 Hld

```

1 struct hld {
2     #define rs(x) resize(x)
3     int n;
4     vector <vector<int>> g;
5     vector <tipo> values, sz, heavy, parent, depth
        , top, r, pos;
6     segtree st; // iterative is faster
7     hld(int _n, vector <vector<int>> _g, vector <
        tipo> _v) {
8         n = _n; g = _g; values = _v;
9         sz.rs(n); parent.rs(n); depth.rs(n); top.rs(n)
            ; pos.rs(n);
10        heavy.rs(n);
11        forn(i,n) heavy[i] = i;
12        dfs(0);
13        get_order(0,0);
14        st.build(r,r.size());
15    }
16    ll dfs(int v, int p = -1, int d = 0) {
17        depth[v] = d; parent[v] = p;
18        for(int &u : g[v]) {
19            if(u == p) continue;
20            ll tam = dfs(u,v,d+1);
21            // move heavy node to the begining
22            if(sz[u] > sz[heavy[v]]) heavy[v] = u, swap
                (u,g[v][0]);

```

```

23     sz[v] += tam;
24 }
25     sz[v] += 1;
26     return sz[v];
27 }
28 void get_order(int v, int t, int p = -1) { //
29     // first child is heavy
30     pos[v] = r.size(); top[v] = t; r.pb(values[v]);
31     for(int u : g[v]) {
32         if(u == p) continue;
33         if(u == heavy[v]) get_order(u,t,v);
34         else get_order(u,u,v);
35     }
36 }
37 void op(ll &res, int a, int b) { // Set
38     // operation
39     res = max(res, st.query(pos[a],pos[b]));
40     // ----- for segtree recursive use
41     // -----
42     // res = max(res, st.query(pos[a],pos[b]).ans);
43 }
44 tipo query_hld(int a, int b) {
45     tipo res = NEUT;
46     for(; top[a] != top[b]; b = parent[top[b]]) {
47         if(depth[top[a]] > depth[top[b]]) swap(a, b);
48         op(res,top[b],b);
49     }
50     if(depth[a] > depth[b]) swap(a, b);
51     op(res,a,b);
52     return res;
53 }
54 void update_hld(int p, tipo val) { st.update(
55     pos[p],val); }
56 };

```

10.3 Lca

```

1 const int MAXN = 200005, LOG = 20;
2 struct LCA {
3     int n, root;
4     vector<vector<int>> g;
5     int jmp[MAXN][LOG], depth[MAXN];
6     void lca_dfs(int x) {
7         for(int u : g[x]) {
8             if(u == jmp[x][0]) continue;
9             jmp[u][0] = x; depth[u] = depth[x]+1;
10            lca_dfs(u);
11        }
12    }
13    LCA(int tam, vector<vector<int>> &tree, int r
14        ): n(tam),root(r),g(tree) {
15        depth[root] = 0;
16        memset(jmp,-1,sizeof(jmp)); jmp[root][0]
17            = root;
18        lca_dfs(root);
19        forr(k, 1, LOG){ forn(i, n){
20            if(jmp[i][k-1]<0) jmp[i][k] = -1;
21            else jmp[i][k] = jmp[jmp[i][k-1]][k-1];
22        }
23    }
24 }

```

```

23 int lca(int x, int y){
24     if(depth[x] < depth[y]) swap(x,y);
25     for(int i = LOG-1; i >= 0; i--) {
26         if(depth[x]-(1<<i) >= depth[y]) x =
27             jmp[x][i];
28     }
29     if(x == y) return x;
30     for(int i = LOG-1; i >= 0; i--) {
31         if(jmp[x][i] != jmp[y][i]) x = jmp[x][i], y = jmp[y][i];
32     }
33     return jmp[x][0];
34 }
35 int dist(int x, int y) {
36     return depth[x] + depth[y] - 2 * depth[
37         lca(x,y)];
38 }
39 };

```

10.4 Sm To Large

```

1 struct query { // v: vtx, c: col, idx_ans
2     ll v, c, idx;
3 };
4 vi g[MAXN];
5 vector<query> q[MAXN]; // Queries to answer
6 vi ans(MAXN, -1), color; // Answer to each query
7 unordered_map<ll, ll> cnt[MAXN];
8 int merge(int v, int u){
9     if(SZ(cnt[v]) < SZ(cnt[u])) swap(u, v);
10    for(auto [x, y]: cnt[u]){
11        cnt[v][x] += y;
12    }
13    cnt[u].clear();
14    return v;
15 }
16 void process_queries(int v, int v_repr){
17     for(auto &[_v, c, i]: q[v])
18         ans[i] = SZ(cnt[v_repr]);
19 }
20 int dfs(int v, int p){
21     int v_repr = v;
22     cnt[v][color[v]]++;
23     for(auto u: g[v]){
24         if(u == p) continue;
25         int u_repr = dfs(u, v);
26         v_repr = merge(v_repr, u_repr);
27     }
28     process_queries(v, v_repr);
29     return v_repr;
30 }
31 void solve(int m) {
32     dfs(0,-1);
33     forn(i,m) cout << ans[i] << " ";
34     cout << "\n";
35 }

```